

A

Project Report on

**SafeSurf — A Machine Learning-Based Real-Time
Phishing Website Detector**

Submitted in partial fulfillment of the requirements
for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering

Submitted by

YALLAPRAGADA OMTEJA	(22P31A0539)
KARRI DEEPIKA	(22P31A0517)
DANGETI SAI GAYATRI VISHWESH	(22P31A0508)
ANKIT ROY CHOWDHURY	(22P31A0504)

Under the esteemed supervision of

Mr. Alla Rajendra, M.Tech, (Ph.D)
Assistant Professor



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY (A)

(Approved by AICTE, New Delhi & Affiliated to JNTUK, Kakinada)

Surampalem, East Godavari District

Andhra Pradesh – 533437

2022–2026

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY (A)

(Approved by AICTE, New Delhi & Affiliated to JNTUK, Kakinada)

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the project work entitled **“SafeSurf — A Machine Learning-Based Real-Time Phishing Website Detector”** is a bonafide work carried out by the students **YALLAPRAGADA OMTEJA (22P31A0539), KARRI DEEPIKA (22P31A0517), DANGETI SAI GAYATRI VISHWESH (22P31A0508), ANKIT ROY CHOWDHURY (22P31A0504)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** from Aditya College of Engineering and Technology, Surampalem, during the academic year 2025–2026.

This project work has not been submitted in full or part to any other University or educational institute for the award of any degree or diploma.

PROJECT SUPERVISOR

Mr. Alla Rajendra, M.Tech, (Ph.D)
Assistant Professor

HEAD OF THE DEPARTMENT

Dr. CHAKKA S.V.V.S.N. Murty, M.Tech, Ph.D
Associate Professor

EXTERNAL EXAMINER

DECLARATION

We hereby declare that this project entitled **“SafeSurf — A Machine Learning-Based Real-Time Phishing Website Detector”** has been undertaken by us and this work has been submitted to Department of Computer Science & Engineering, **ADITYA COLLEGE OF ENGINEERING AND TECHNOLOGY (A)**, Surampalem affiliated to JNTUK, Kakinada, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**.

We further declare that this project work has not been submitted in full or part to any other University or educational institute for the award of any degree or diploma.

PROJECT ASSOCIATES

YALLAPRAGADA OMTEJA (22P31A0539)

KARRI DEEPIKA (22P31A0517)

DANGETI SAI GAYATRI VISHWESH (22P31A0508)

ANKIT ROY CHOWDHURY (22P31A0504)

ACKNOWLEDGEMENT

It is with immense pleasure that we would like to express our indebted gratitude to our project supervisor, Mr. Alla Rajendra, M.Tech, (Ph.D), who has guided us a lot and encouraged us in every step of project work. His valuable moral support and guidance has been helpful in successful completion of this project.

We wish to express our sincere thanks to Dr. CHAKKA.S.V.V.S.N.Murthy, M.Tech., Ph.D., Head of the Department of CSE, for his valuable guidance given to us throughout the period of the project work.

We feel elated to thank Principal, Dr. A. RAMESH, M.Tech., Ph.D., of Aditya College of Engineering and Technology for his cooperation in completion of our project and throughout our course.

We feel elated to thank Dr. Pullela S.V.V.S.Ravi Kumar, M.Tech., Ph.D., Dean (Academics) of Aditya College of Engineering and Technology (A) for his cooperation in completion of our project work.

We wish to express our sincere thanks to all faculty members, and lab programmers for their valuable guidance given to us throughout the period of the project.

We avail this opportunity to express our deep sense and heart full thanks to the Management of Aditya College of Engineering & Technology (A) for providing a great support for us by arranging the trainees, and facilities needed to complete our project and for giving us the opportunity for doing this work.

PROJECT ASSOCIATES

YALLAPRAGADA OMTEJA (22P31A0539)

KARRI DEEPIKA (22P31A0517)

DANGETI SAI GAYATRI VISHWESH (22P31A0508)

ANKIT ROY CHOWDHURY (22P31A0504)



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

An AUTONOMOUS Institution

Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade

Recognized by UGC under Sections 2(f) and 12(B) of UGC Act, 1956

Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.

Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

VISION

To induce higher planes of learning by imparting technical education with

- ✓ International standards
- ✓ Applied research
- ✓ Creative Ability
- ✓ Value based instruction and to emerge as a premiere institute

MISSION

Achieving academic excellence by providing globally acceptable technical education by forecasting technology through

- ✓ Innovative Research And development
- ✓ Industry Institute Interaction
- ✓ Empowered Manpower



Principal
PRINCIPAL
Aditya College of
Engineering & Technology
SURAMPALEM- 533 437



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

An AUTONOMOUS Institution

Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade
Recognized by UGC under Sections 2(F) and 12(B) of UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.
Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

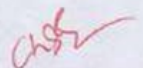
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION:

To become a center for excellence in Computer Science and Engineering education and innovation.

MISSION:

- Provide state of art infrastructure
- Adapt skill-based learner centric teaching methodology
- Organize socio cultural events for better society
- Undertake collaborative works with academia and industry
- Encourage students and staff self-motivated, problem-solving individuals using Artificial Intelligence
- Encourage entrepreneurship in young minds.


Head of the Department
Head of the Department
Dept. of CSE
Aditya College of Engineering
& Technology
SURAMPALAM-533437



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

An AUTONOMOUS Institution

Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade
Recognized by UGC under Sections 2(f) and 12(B) of UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.
Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROGRAM EDUCATIONAL OBJECTIVES

- PEO1: Capability to design and develop new software products as per requirements of the various domains and eligible to take the roles in various government, research organizations and industry
- PEO2: More enthusiastic to adopt new technologies and to improve existing solutions by reducing complexity which serves society requirements as per timeline changes
- PEO3: With good hands-on basic knowledge and ready improve academic qualifications in India or Abroad
- PEO4: Ability to build and lead the team to achieve organization goals.


Head of the Department
Dept. of CSE
Aditya College of Engineering
& Technology
SURAMPALEM-533437



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

An **AUTONOMOUS** Institution

Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade

Recognized by UGC under Sections 2(F) and 12(B) of UGC Act, 1956

Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.

Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

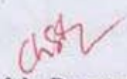
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROGRAM SPECIFIC OUTCOMES

PSO 1: The ability to design and develop computer programs for analyzing the data.

PSO 2: The ability to analyze data & develop Innovative ideas and provide solution by adopting emerging technologies for real time problems of software industry.

PSO 3: To encourage the research in software field that contribute to enhance the standards of human life style and maintain ethical values.


Head of the Department
Head of the Department
Dept. of CSE
Aditya College of Engineering
& Technology
SURAMPALAM, 533 437



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

An AUTONOMOUS Institution

Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade

Recognized by UGC under Sections 2(F) and 12(B) of UGC Act, 1956

Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.

Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROGRAM OUTCOMES

- 1. ENGINEERING KNOWLEDGE:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- 2. PROBLEM ANALYSIS:** Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- 3. DESIGN/DEVELOPMENT OF SOLUTIONS:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- 4. CONDUCT INVESTIGATIONS OF COMPLEX PROBLEMS:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- 5. MODERN TOOL USAGE:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
- 6. THE ENGINEER AND SOCIETY:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues, and the consequent responsibilities relevant to the professional engineering practice.
- 7. ENVIRONMENT AND SUSTAINABILITY:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- 8. ETHICS:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- 9. INDIVIDUAL AND TEAM WORK:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- 10. COMMUNICATION:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, give and receive clear instructions.
- 11. PROJECT MANAGEMENT AND FINANCE:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- 12. LIFE-LONG LEARNING:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

CHB
Head of the Department
Head of the Department
Dept. of CSE
Aditya College of Engineering
& Technology
SURAMPALEM-533437

ABSTRACT

SafeSurf — A Machine Learning-Based Real-Time Phishing Website Detector

Phishing attacks remain a significant cybersecurity threat, targeting users through deceptive websites that mimic legitimate platforms to steal sensitive information. Rapid and accurate detection of such malicious websites is therefore essential for ensuring safe on-line interactions. This project presents **SafeSurf**, a real-time phishing website detection system based on supervised machine learning techniques. The system analyzes multiple URL and website characteristics—including domain attributes, HTTPS usage, URL length, and structural HTML indicators—to effectively distinguish between phishing and legitimate websites.

Unlike traditional blacklist-based approaches, which often fail to detect newly generated phishing domains, **SafeSurf** employs data-driven learning methods trained on a dataset of phishing and benign URLs. The system incorporates preprocessing techniques such as handling missing values, feature normalization, and noise reduction to improve model reliability. Feature importance analysis is used to identify key indicators associated with phishing activity, while an ensemble-based machine learning approach combines multiple classification models to improve detection performance and robustness.

Experimental results show that the proposed system achieves high detection accuracy ($> 97\%$) with efficient prediction time, making it suitable for real-time applications. By integrating machine learning with automated feature analysis, **SafeSurf** provides a scalable and adaptive solution for phishing detection and helps strengthen cybersecurity by reducing users' exposure to malicious websites.

Mentor: Mr. Alla Rajendra

Batch No: 16

Batch Members:

YALLAPRAGADA OMTEJA (22P31A0539)

KARRI DEEPIKA (22P31A0517)

DANGETI SAI GAYATRI VISHWESH (22P31A0508)

ANKIT ROY CHOWDHURY (22P31A0504)

Guide Signature

INDEX

ABSTRACT	x
1 Introduction	1
1.1 Purpose of the Project	1
1.2 Scope of the Project	1
1.3 Current System	2
1.4 Proposed System	2
2 Requirement Analysis	4
2.1 Functional Requirements	4
2.1.1 Core Functional Behavior of SafeSurf	4
2.2 Non-Functional Requirements	7
2.2.1 SafeSurf System Constraints and Quality Attributes	7
3 System Analysis	10
3.1 Introduction	10
3.2 Use Cases & Actors	10
3.2.1 Actors	10
3.2.2 Core Use Cases	11
3.3 Algorithms Used	13
3.3.1 DNS Guard (Deterministic DNS Resolution)	13
3.3.2 Multi-Model Voting Classifier Ensemble	13
3.3.3 Natural Language Processing - Bag of Words	14
3.3.4 Weighted Soft-Voting Merger (Fusion Algorithm)	15
3.4 System Flow Chart	15
4 System Design	17
4.1 Introduction to System Design	17
4.2 System Architecture	17
4.2.1 Client Layer	17
4.2.2 Processing & Detection Layer	18
4.2.3 Response Layer	18
4.3 System Components	20
4.3.1 URL Input & Preprocessing	20

4.3.2	DNS Guard Layer	20
4.3.3	Feature Extraction Engine	20
4.3.4	Machine Learning Engine	21
4.3.5	Decision Engine	21
4.3.6	Response Generation	21
4.4	Backend & Model Implementation	21
4.4.1	FastAPI Backend	21
4.4.2	Hybrid Ensemble Model	22
4.5	Frontend & UI Design	22
4.6	Model Design & Optimization	23
4.6.1	Feature Engineering	23
4.6.2	Hybrid Ensemble Model Fusion	23
4.7	Development Workflow	24
4.8	Insights	24
5	Implementation	25
5.1	Implementation Resources	25
5.1.1	Project Directory Structure	25
5.1.2	Implementation Environment	26
5.2	System Requirements	27
5.2.1	Basic Requirements	27
5.2.2	Advanced Requirements (For Extensive Hyperparameter Tuning & Model Optimization)	28
5.2.3	How It Works ?	28
5.3	Dataset Description	29
5.3.1	Core Data Sources	30
5.3.2	Dataset Overview	30
5.3.3	111-Attribute Deep Lexical & Infrastructure Model (The Complete Feature Set)	33
5.4	Web Framework, Frontend, and Deployment Technologies Used	33
5.4.1	FastAPI (Python Web Framework) Introduction to FastAPI	33
5.4.2	REST API (Representational State Transfer API) Introduction to REST API	34
5.4.3	React & Vite – Frontend Framework Introduction to React & Vite	35
5.4.4	Tailwind CSS – Frontend Styling Introduction to Tailwind CSS	36
5.4.5	Docker – Containerization and Deployment Introduction to Docker	37
5.5	Programming Language Used	38
5.5.1	Introduction to Python	38

5.5.2	Python Version and Environment	40
5.5.3	Python's Role in Machine Learning and Feature Engineering	41
5.5.4	Deployment of External and Internal APIs	42
5.5.5	Insights	43
5.6	Libraries/Packages Used	43
5.6.1	NumPy (Numerical Python)	44
5.6.2	Pandas (Data Analysis and Manipulation Library)	44
5.6.3	Matplotlib & Seaborn (Data Visualization Libraries)	45
5.6.4	Scikit-learn (Machine Learning Library)	46
5.6.5	XGBoost (Extreme Gradient Boosting Library)	46
5.6.6	NLTK (Natural Language Toolkit)	47
5.6.6.1	Why was NLTK used in this research?	47
5.6.7	Insights	47
5.7	Source Code	49
5.7.1	Concurrent Infrastructure Extraction	49
5.7.2	In-Memory Rolling TTL Caching	49
5.7.3	Weighted Ensemble Fusion	50
5.7.4	Comprehensive DNS Signal Extraction	50
5.7.5	Production Drift Guard	51
5.7.6	NXDOMAIN Pre-Check (DNS Guard)	51
5.7.7	Robust URL Normalization	51
5.7.8	Operational Health & Readiness	52
5.7.9	Heuristic Suspicion Boosting	52
5.7.10	Primary Analysis API Endpoint	53
5.7.11	SSL Handshake Validation	53
5.7.12	Centralized App Configuration	54
5.7.13	Live Model Hot-Reloading	54
5.7.14	WHOIS Age Calculation	55
6	Testing	56
6.1	Introduction	56
6.2	Dataset Analysis	56
6.3	Three-Phase Retraining and Validation Workflow	57
6.4	Model Performance Observations	58
6.5	Test Case Identification and Results	58
6.6	Impact of System Components on Stability	61
6.7	Final Insights	61

7	Result Analysis	62
7.1	Introduction	62
7.2	Model Evaluation Metrics	62
7.3	Component-Level Results	63
7.3.1	DNS Guard Efficiency	63
7.3.2	Feature Extraction Speed	64
7.3.3	Hybrid Ensemble Effectiveness	64
7.4	Threshold Logic Validation	64
7.5	Performance Comparison	66
8	Conclusion	68
9	Future Scope	69
10	Bibliography	70
11	Annexure	72
11.1	Base Paper	72

LIST OF FIGURES

3.1	System Use Case Diagram	11
3.2	Chrome Extension Real-Time Workflow	12
3.3	DNS Guard	13
3.4	Multi-Model Voting Classifier Ensemble	14
3.5	Natural Language Processing	14
3.6	Weighted Soft-Voting Merger	15
3.7	System Flowchart	16
4.1	System Architecture	18
4.2	Sequence Diagram	19
4.3	System Component Architecture	23
4.4	Workflow	24
7.1	Confusion Matrix & ROC-plot	63
7.2	NXDOMAIN	63
7.3	Safe Website Detection Using Dashboard	65
7.4	Phishing Website Detection Using Dashboard	65
7.5	Safe & Risk Detection Using Extension	66
7.6	Performance Comparison Against Baseline Models (Multi-Metric Analysis)	66

LIST OF TABLES

5.1	Selected Key Attributes for Phishing Detection	32
6.1	System Validation Summary Table	60
7.1	Comparative Performance Analysis of Evaluated Models	67

LIST OF CODE BLOCKS

5.1	Concurrent Infrastructure Signal Extraction (deep_feature_extractor.py)	49
5.2	Rolling Time-Bucket Caching Mechanism (deep_feature_extractor.py)	49
5.3	Weighted Soft-Voting Ensemble Fusion (ensemble_service.py)	50
5.4	DNS Structural Signal Extraction Pipeline (deep_feature_extractor.py)	50
5.5	Statistical Feature Drift Guard (xgb_service.py)	51
5.6	NXDOMAIN Pre-Check Domain Verification (dns_guard.py)	51
5.7	Canonical URL Normalization Pipeline (url_normalizer.py)	52
5.8	Service Health and Readiness Monitoring (predict.py)	52
5.9	Heuristic Suspicion Boosting Logic (ensemble_service.py)	53
5.10	FastAPI Core Analysis Router Endpoint (predict.py)	53
5.11	Live SSL Handshake Trust Verification (deep_feature_extractor.py)	54
5.12	Centralized Pydantic Configuration System (config.py)	54
5.13	Runtime Model Hot-Reloading Logic (predict.py)	55
5.14	Domain Age Calculation from WHOIS Registry (whois_service.py)	55

1. INTRODUCTION

1.1 Purpose of the Project

Our project, **SafeSurf**, aims to develop a real-time phishing website detection system using advanced machine learning techniques [1]. The system analyzes URL and website infrastructure characteristics to identify and classify malicious phishing websites before users interact with them. By combining automated feature extraction with a multi-model ensemble learning approach, **SafeSurf** provides accurate and reliable phishing detection for modern web environments.

The primary goal of the project is to enhance cybersecurity by enabling proactive detection of phishing attacks that attempt to steal sensitive user information such as login credentials, financial data, and personal details. The system is designed for internet users, organizations, and cybersecurity platforms to improve online safety and reduce exposure to malicious websites. Key benefits of **SafeSurf** include real-time detection, high classification accuracy, minimal user interaction, and automated threat analysis.

1.2 Scope of the Project

The scope of **SafeSurf** involves designing and implementing an intelligent phishing detection framework capable of analyzing URLs and website infrastructure data in real time [2–4]. The system integrates machine learning models, infrastructure analysis, and browser-based detection mechanisms to provide effective protection against phishing threats. The system consists of the following components:

- **Data Collection** – Aggregation of phishing and legitimate URL datasets, primarily the PhiUSIIL dataset [5], from public sources for training and evaluation.
- **Feature Extraction** – Analysis of domain and infrastructure characteristics such as DNS records, SSL certificate information, domain registration details, URL structure, and HTML patterns.
- **Machine Learning Model** – Implementation of a VotingClassifier ensemble combining multiple models including Logistic Regression, LinearSVC, Random Forest, HistGradientBoosting, and Calibrated XGBoost to improve prediction accuracy.
- **Detection Architecture** – A layered detection pipeline where a DNS Guard module filters invalid domains, followed by asynchronous infrastructure analysis using

DNS, SSL, and RDAP information extraction.

- **User Interface and Browser Extension** – Development of a web-based dashboard and Chrome extension that intercept URLs and provide users with a clear **Safe** or **Phishing** verdict in real time.

The system is designed to support applications in **browser security tools, enterprise cybersecurity systems, and web protection platforms**. Future enhancements may include large-scale cloud deployment, integration with threat intelligence feeds, and automated browser protection mechanisms.

1.3 Current System

Existing phishing detection systems primarily rely on blacklist-based approaches, rule-based filters, or simple machine learning models to identify malicious websites [6–10]. While these systems provide a basic level of protection, they often struggle to detect newly generated phishing websites and rapidly evolving attack techniques. Many current approaches:

- Depend heavily on static blacklists that require continuous manual updates.
- Have limited capability to detect zero-day phishing domains that have not yet been reported.
- Use single-model classifiers, which may reduce detection accuracy and robustness.
- Lack real-time infrastructure analysis, preventing deeper inspection of domain behavior and hosting characteristics.

As phishing techniques become more sophisticated, traditional detection mechanisms are often insufficient to provide reliable protection against emerging threats.

1.4 Proposed System

The proposed system, **SafeSurf**, introduces an advanced phishing detection framework that combines infrastructure analysis with a machine learning ensemble approach to improve detection accuracy and reliability [11].

The system implements a layered detection architecture beginning with a deterministic DNS Guard module that filters invalid domain lookups. This is followed by concurrent infrastructure data extraction using asynchronous processing to analyze DNS records, SSL certificates, and RDAP domain registration information.

SafeSurf employs a highly optimized VotingClassifier ensemble model, integrating multiple machine learning algorithms including Logistic Regression, LinearSVC, Random Forest, HistGradientBoosting, and Calibrated XGBoost. By combining predictions from multiple classifiers, the system improves both precision and recall when identifying phishing websites, including previously unseen or zero-day threats.

To provide practical usability, **SafeSurf** includes a web-based dashboard and Chrome browser extension that intercept URLs and automatically displays a clear detection verdict—either **Safe** or **Phishing**. This design ensures minimal user interaction while maintaining real-time protection.

The system is designed with key objectives including real-time detection, scalability, high precision, and minimal user intervention. In the future, **SafeSurf** can be extended with cloud-based deployment, integration with global threat intelligence sources, and automated browser-level security mechanisms, enabling more robust protection against evolving phishing attacks.

2. REQUIREMENT ANALYSIS

2.1 Functional Requirements

Functional requirements define the core operations and capabilities of the **SafeSurf** system. The system accepts input URLs, processes them through a multi-layered machine learning pipeline, and produces phishing detection results. The goal is to ensure accurate, real-time classification with minimal latency while maintaining a simple and user-friendly interface.

2.1.1 Core Functional Behavior of SafeSurf

User Interface & Interaction

- SafeSurf provides a React.js-based web application that allows users to manually enter and analyze URLs, along with a Chrome extension that continuously monitors URLs in the background during browsing sessions.
- Users are able to input URLs directly through the web dashboard or rely on the Chrome extension to automatically scan URLs without requiring manual intervention.
- The system returns a detailed response that includes the classification result (**Safe, Phishing, or Invalid Domain**) along with a clearly defined risk level categorized as **Low, Medium, or High**.
- The user interface is designed to be simple, intuitive, and responsive, ensuring that users can easily understand results without being exposed to internal model complexities or raw probability values.

URL Processing Pipeline

- The system accepts a URL as input and performs normalization steps such as removing unnecessary ports, handling trailing slashes, and ensuring consistent formatting across different inputs.
- Input validation and sanitization mechanisms are applied to detect malformed or potentially harmful inputs before further processing.

- A DNS Guard layer performs domain resolution checks, identifies non-existent domains (NXDOMAIN), and filters out invalid URLs before they reach the machine learning pipeline.
- This preprocessing stage reduces unnecessary computation and improves system efficiency by ensuring that only valid URLs proceed to feature extraction and prediction.

Feature Extraction Engine

- The system extracts a total of 111 features from the input URL, covering lexical characteristics, network-level information, and textual patterns.
- Lexical features include properties such as URL length, structure, number of special characters, and token distribution within the URL.
- Network features include DNS records (A, NS, MX), WHOIS registration details, domain age, and SSL certificate-related information.
- NLP-based features are generated using a Bag-of-Words representation to capture textual patterns within URLs.
- Feature extraction is implemented using asynchronous and parallel processing techniques (such as `asyncio.gather`) to reduce latency and efficiently handle network-bound operations.

Machine Learning Detection Engine

- The system uses an ensemble of machine learning models including Logistic Regression, Linear Support Vector Machine (LinearSVC), Random Forest, Histogram Gradient Boosting, and a calibrated XGBoost classifier.
- A separate NLP-based Logistic Regression model is used to analyze textual characteristics of URLs independently of infrastructure-based features.
- The outputs from these models are combined using a weighted soft-voting approach, where infrastructure-based predictions contribute 65% and text-based predictions contribute 35% to the final result.
- This hybrid approach improves overall prediction accuracy by leveraging both structural and textual characteristics of URLs.

Real-Time Detection & Response

- The system processes incoming requests in real time using a FastAPI backend that supports asynchronous request handling and efficient concurrency.
- The API returns a structured JSON response containing the classification result, risk level, and additional metadata derived from the prediction pipeline.
- Risk levels are determined using predefined probability thresholds, where values above a certain limit (e.g., 0.85) are categorized as high risk.
- The Chrome extension consumes this response to notify users and prevent access to high-risk websites during browsing.

System Maintainability

- The system follows a modular architecture in which machine learning models, feature extraction components, and API layers are separated for easier updates and maintenance.
- This modular design allows independent updates to models and feature pipelines without affecting the overall system.
- The system supports version control using Git and allows retraining and calibration of models as new data becomes available.

Software Requirements

- **Frontend:**
 - React.js for the responsive web interface [12].
 - Chrome Extension APIs for background script processing.
- **Backend:**
 - FastAPI (Python 3.9+) for core API architecture and machine learning pipeline integration [13].
- **Machine Learning & Data Processing:**
 - Scikit-learn (VotingClassifier, NLP Models) [14].
 - XGBoost (Infrastructure Intelligence) [15, 16].
 - NumPy and Pandas for data manipulation.

- **Deployment:**

- Docker for multi-stage containerization [17].

Hardware Requirements

- **Minimum Runtime Requirements:**

- Processor: Intel i5 / Ryzen 5 or equivalent.
- RAM: 8 GB (16 GB recommended).

- **Training Environment Recommended:**

- RAM: 16 GB+ for parallel feature extraction across 200k+ dataset rows.
- Storage: SSD with at least 15 GB of free space for datasets and generated CSV files.

2.2 Non-Functional Requirements

Non-functional requirements define the performance, scalability, security, and reliability characteristics of the SafeSurf system.

2.2.1 SafeSurf System Constraints and Quality Attributes

Performance

- The system is designed to provide responses within an average time of less than 500 milliseconds under normal operating conditions.
- It supports real-time processing of URLs with optimized feature extraction and prediction pipelines.
- External network operations such as DNS and WHOIS queries are controlled using timeout limits (e.g., 15 seconds) to prevent blocking.

Scalability

- The system uses concurrency control mechanisms such as semaphores to efficiently manage multiple simultaneous requests.
- It supports horizontal scaling through containerized deployment and cloud infrastructure.

- Future enhancements include caching mechanisms (e.g., Redis) to improve response time for frequently accessed URLs.

Availability

- The system targets an uptime of approximately 99.8% and is designed to handle partial failures gracefully.
- Failures from external services such as DNS resolution, WHOIS lookups, or SSL verification are handled without crashing the system.
- The system supports dynamic model reloading (`/reload-model`) without requiring a full restart, ensuring continuous availability.

Security

- Input validation and sanitization are applied to all incoming URLs to prevent malformed or malicious inputs.
- Request size limits are enforced using middleware (e.g., `MAX_REQUEST_BODY_BYTES`) to prevent large payload attacks.
- All communication between client and server is secured using HTTPS.
- Cross-Origin Resource Sharing (CORS) policies are configured to restrict access to trusted sources only.

Reliability

- The system maintains a prediction accuracy greater than 97% based on trained models.
- It ensures consistent outputs across different datasets and input variations.
- The system is designed to remain stable under varying workloads and concurrent usage conditions.

Usability

- The interface is designed to be easy to understand and accessible to non-technical users.
- Results are presented in a clear and concise format without exposing internal technical details.

- The Chrome extension operates automatically, requiring minimal user interaction.

Maintainability

- The system follows a clean and modular code structure separating API routes, feature extraction logic, and model inference.
- Configuration settings are managed using environment variables (`.env`) for flexibility.
- Logging and monitoring mechanisms can be integrated to track system performance and errors.

Deployment & Supportability

- The application is containerized using Docker for consistent deployment across environments.
- It is compatible with cloud platforms and supports scaling based on demand.
- The system is designed to integrate with CI/CD pipelines for automated deployment and updates.

3. SYSTEM ANALYSIS

3.1 Introduction

SafeSurf is a production-grade, infrastructure-aware Machine Learning system explicitly designed for real-time phishing website detection [1]. The system provides definitive, actionable intelligence (Safe/Phishing) by analyzing Uniform Resource Locators (URLs) subjected to a layered, deterministic, and probabilistic pipeline.

The core of SafeSurf combines a fast deterministic pre-check (DNS Guard) with a robust Multi-Model Voting Classifier Ensemble and Natural Language Processing (NLP). The architecture leverages 111 distinct features spanning lexical structure, domain age, SSL certificate details, and DNS infrastructure information to achieve a highly accurate detection capability ($> 97\%$ accuracy). Users interact with the system via a standalone Visual Security Dashboard and a real-time protective browser extension.

3.2 Use Cases & Actors

3.2.1 Actors

1. **End User / Web Surfer:** The primary consumer of the system. They browse the internet and rely on the SafeSurf system to protect them from malicious websites. They interact with the system primarily through the Chrome extension and the web dashboard [18].
2. **System Administrator / Security Analyst:** Responsible for managing the deployed application, monitoring real-time performance metrics via the dashboard, configuring .env thresholds, and actively re-training the ML models with updated datasets.
3. **SafeSurf API (System Actor):** The FastAPI backend system that processes incoming requests, orchestrates the layered detection pipeline, interfaces with the ML models, and returns unified JSON risk analysis responses [13].
4. **External Infrastructure (System Actors):** Third-party services, including DNS Resolvers, WHOIS registry databases, and SSL authorities. The system queries these actors to accumulate infrastructure footprint data for a given URL.

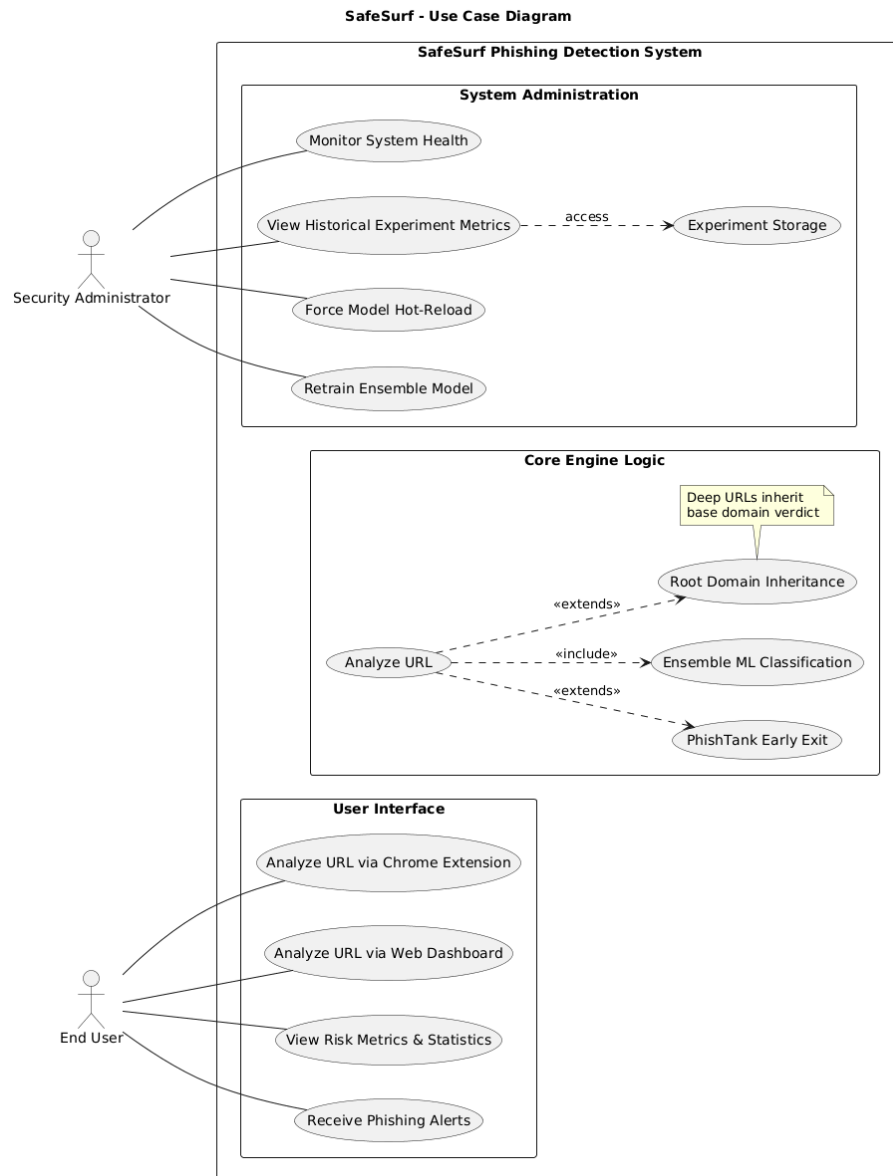


Figure 3.1: System Use Case Diagram

3.2.2 Core Use Cases

1. **Analyze URL Safety:** An End User inputs a URL manually (or automatically via the extension). The system processes the URL and returns a definitive "safe", "phishing", or "invalid" verdict, mapping it to a risk level (LOW/MEDIUM/HIGH).
2. **Model Re-training & Evaluation:** The System Analyst utilizes the three-phase data pipeline to extract features from a raw CSV dataset, retrain the multi-model ensemble, and hot-reload the .pkl files into the live detection server without down-time.
3. **Metric Dashboarding:** The End User or Analyst logs onto the web interface to

view the system’s historical detection accuracy rate, precision metrics, and ROC-AUC visualizations directly extracted from the metrics.json file.

4. **Automated Browser Protection:** The Chrome Extension passively captures URLs visited by the End User, querying the API in the background. If a threshold is exceeded, it alerts or blocks the user from proceeding to the malicious site.

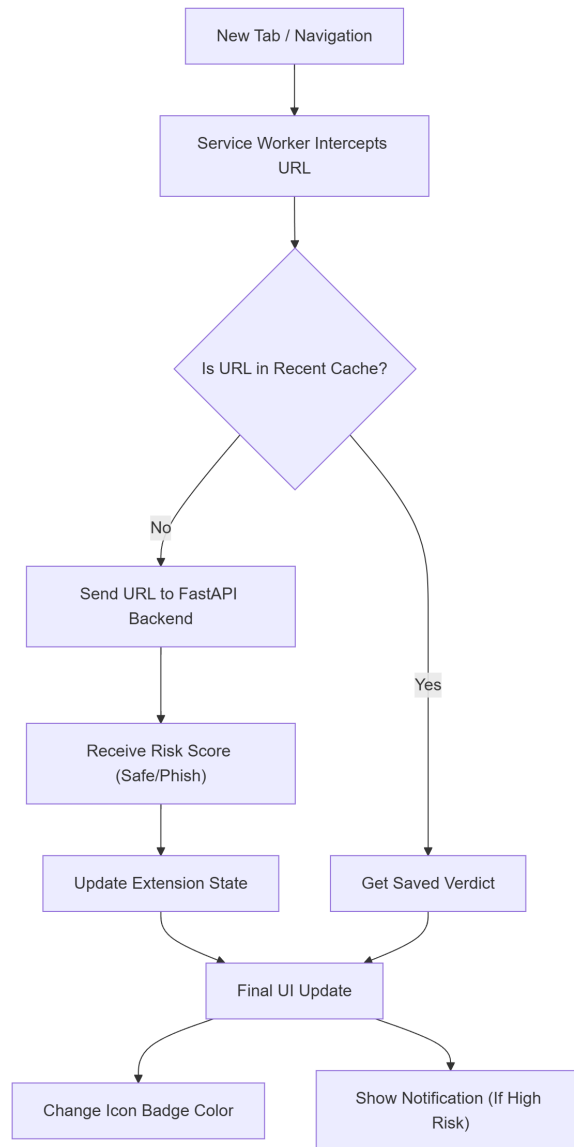


Figure 3.2: Chrome Extension Real-Time Workflow

3.3 Algorithms Used

The detection engine utilizes a hybrid approach, merging deterministic heuristics with probabilistic machine learning models [19].

3.3.1 DNS Guard (Deterministic DNS Resolution)

A pre-check algorithm ensuring the URL actually exists. It attempts to resolve the domain's A-record. If it returns NXDOMAIN (No Answer), the algorithm halts the pipeline and instantly flags the URL as "invalid" (High Risk), significantly saving computational resources.

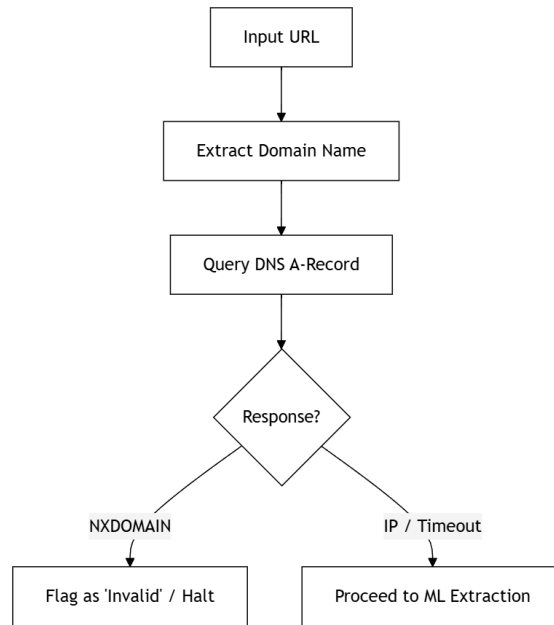


Figure 3.3: DNS Guard

3.3.2 Multi-Model Voting Classifier Ensemble

A robust ensemble algorithm utilized for scikit-learn compatibility and overall generalization [14]. It analyzes 111 deep structural features (character counts, DNS TTL, SSL properties, WHOIS timing) derived from comprehensive phishing datasets [5]. The Voting Classifier internally aggregates predictions from:

- Logistic Regression
- Linear Support Vector Classifier (LinearSVC)
- Random Forest Classifier
- Histogram-based Gradient Boosting Classifier

- eXtreme Gradient Boosting (XGBoost) [15, 16]

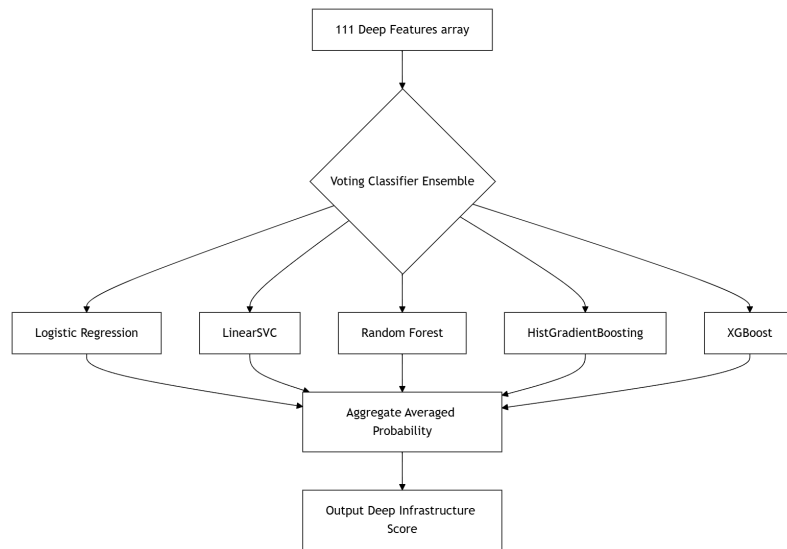


Figure 3.4: Multi-Model Voting Classifier Ensemble

3.3.3 Natural Language Processing - Bag of Words

An algorithm focusing solely on the lexical shape and text strings within the URL. It utilizes a CountVectorizer [14] to generate token arrays and scores the visual characteristics using a LogisticRegression model.

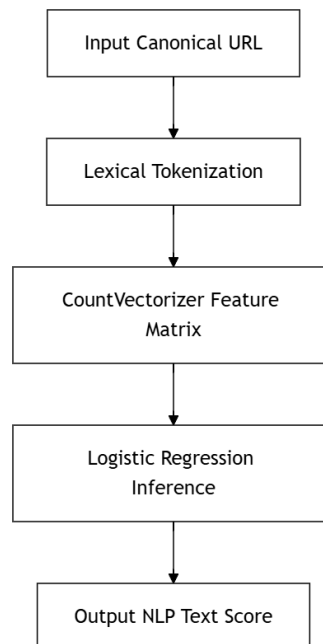


Figure 3.5: Natural Language Processing

3.3.4 Weighted Soft-Voting Merger (Fusion Algorithm)

The overarching algorithm that merges the predictions from the Deep Infrastructure model and the NLP Text model. It computes a mathematically weighted probability to form the final verdict.

$$\text{Final Probability} = (0.65 \times \text{XGB_Ensemble_Score}) + (0.35 \times \text{NLP_Score})$$

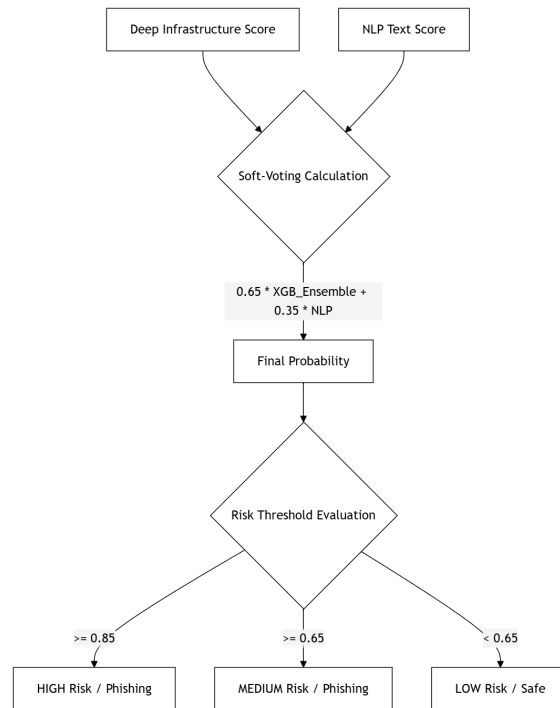


Figure 3.6: Weighted Soft-Voting Merger

3.4 System Flow Chart

The following flowchart illustrates the step-by-step pipeline from URL ingestion to the final risk classification.

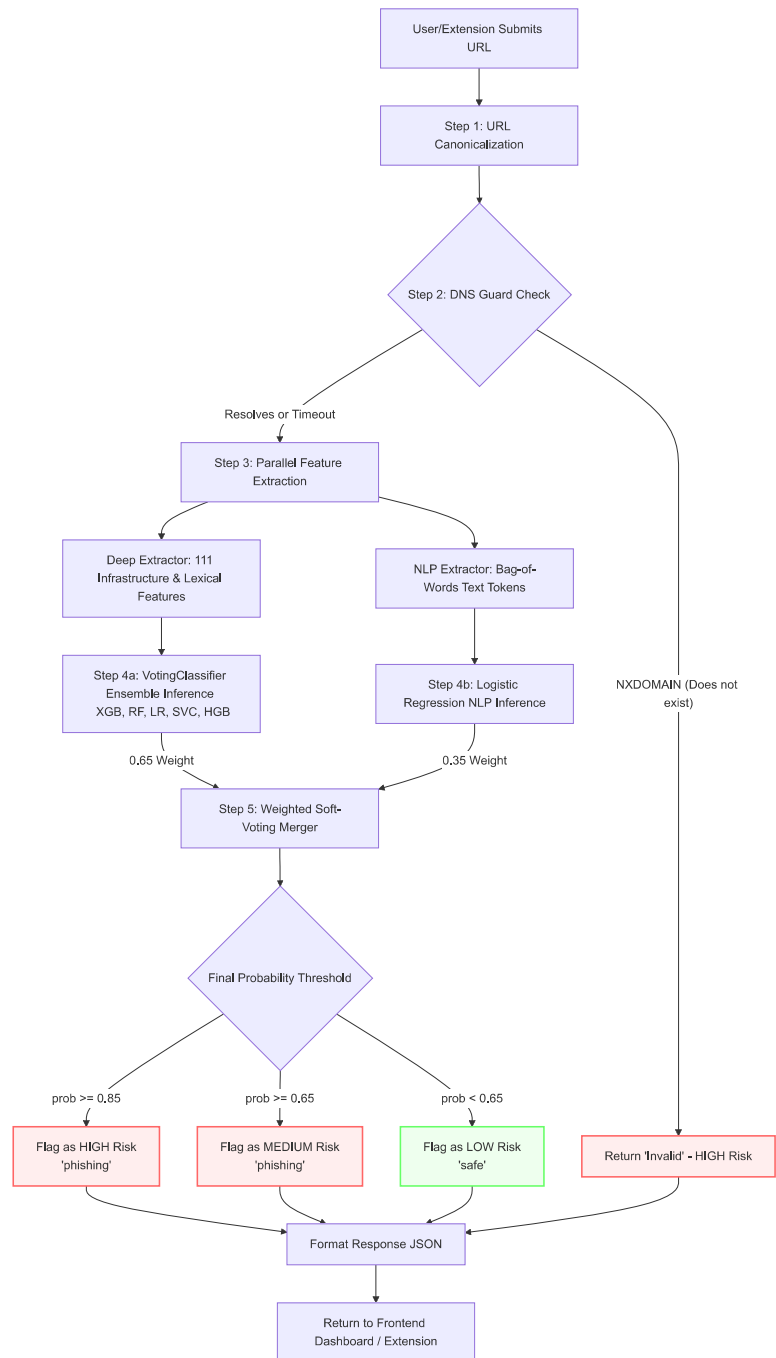


Figure 3.7: System Flowchart

4. SYSTEM DESIGN

4.1 Introduction to System Design

The SafeSurf system is designed as a machine learning–based real-time phishing detection system that integrates a modern web interface, browser extension, and a scalable backend architecture. The system follows a modular and layered design, consisting of:

- React-based frontend and Chrome extension for user interaction
- FastAPI backend for request handling and model execution [13]
- Hybrid ensemble machine learning model for phishing detection [14]

Unlike traditional systems, SafeSurf incorporates:

- DNS Guard layer for deterministic early filtering
- Asynchronous feature extraction (111 infrastructure and lexical features)
- Hybrid ensemble (VotingClassifier + NLP Text model)

This architecture ensures:

- Real-time detection with low latency
- High accuracy (exceeding 97%)
- Efficient computational resource utilization

4.2 System Architecture

4.2.1 Client Layer

- Chrome Extension (real-time background detection)
- React Web Dashboard UI (manual URL analysis and metrics)
- Displays straightforward verdicts:
 - Safe
 - Phishing Risk

4.2.2 Processing & Detection Layer

- FastAPI Backend Engine
- URL Canonicalization
- Semaphore Concurrency Control
- DNS Guard (deterministic NXDOMAIN gate)
- Feature Extraction (111 parallel infrastructure features)
- Hybrid ML Ensemble Engine (Infrastructure ML + Linguistic NLP)

4.2.3 Response Layer

- Soft-Voting Merger & Probability Threshold Engine
- Structured JSON Response Builder
- Unified Risk Level mapping (HIGH / MEDIUM / LOW)

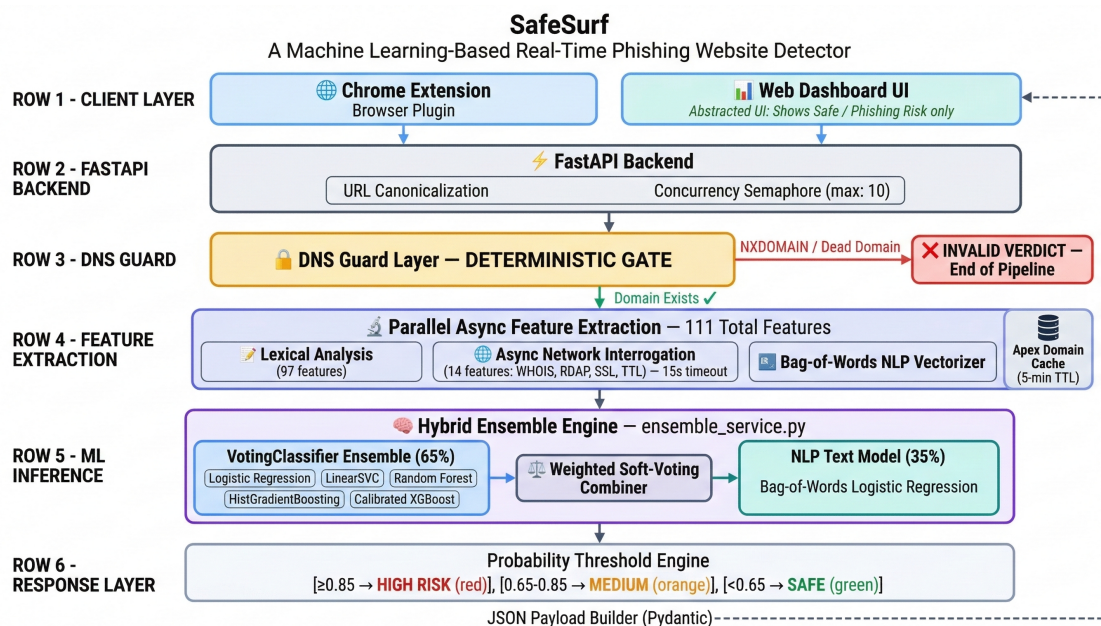


Figure 4.1: System Architecture

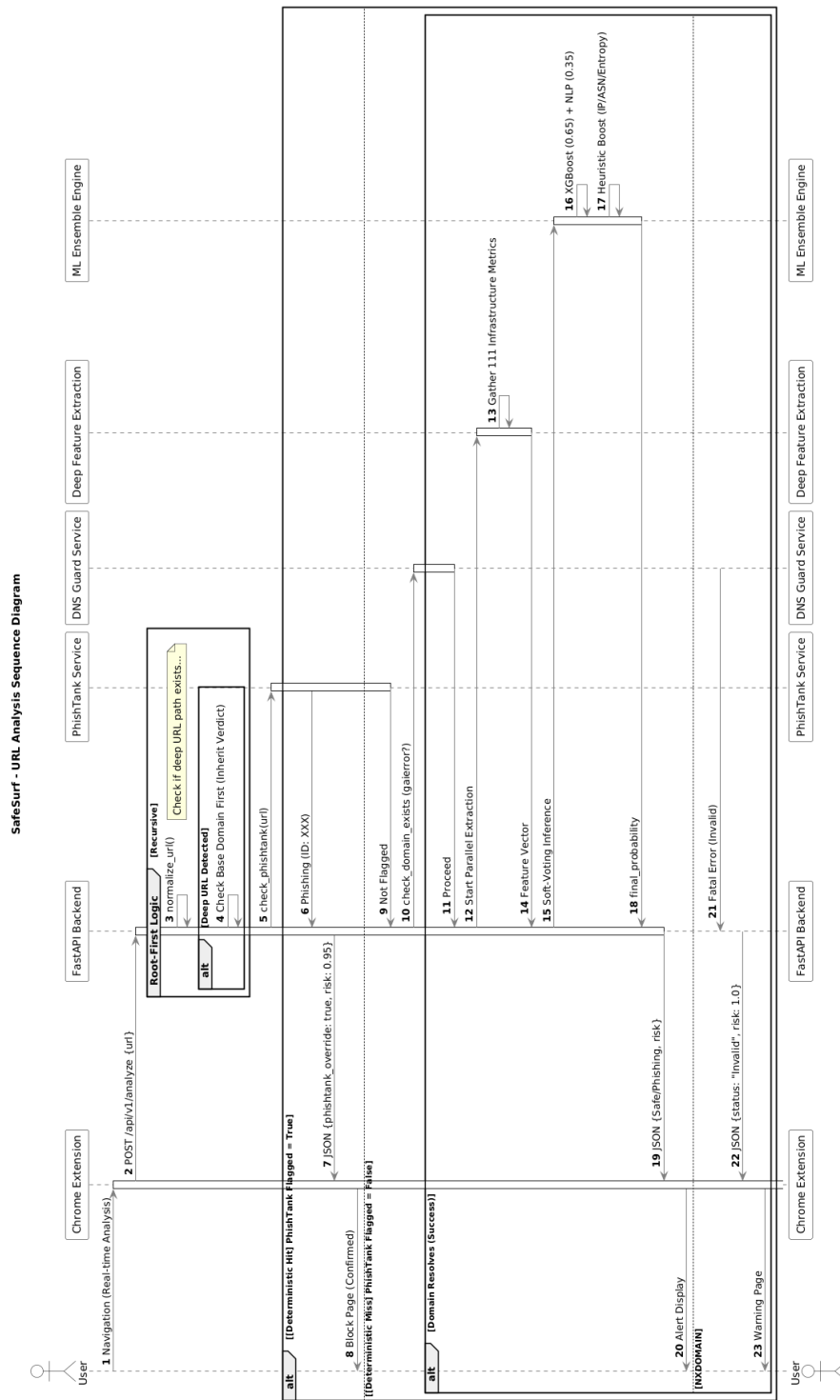


Figure 4.2: Sequence Diagram

4.3 System Components

4.3.1 URL Input & Preprocessing

- Captures URL from:
 - Chrome extension passively navigating
 - Web interface direct manual input
- Performs:
 - URL normalization (Canonicalization: stripping default ports, collapsing slashes)
 - Malformed input validation

4.3.2 DNS Guard Layer

- Acts as the first deterministic security filter
- Blocks:
 - NXDOMAIN (Non-existent domains)
 - Dead or unreachable infrastructure
- Instantly halts the request, drastically reducing unnecessary ML computation [3,4]

4.3.3 Feature Extraction Engine

- Extracts exactly 111 deep features:
 - Lexical features (URL structure lengths, character counts, symbol frequencies)
 - Infrastructure/Network features (WHOIS timing, DNS TTL/Records, SSL certificate validation)
 - NLP features (Bag-of-Words text token representation)
- Leverages:
 - Parallel asynchronous processing (`asyncio.gather`)
 - 15-second strict timeout handling

4.3.4 Machine Learning Engine

- Specialized Hybrid ensemble system:
 - Multi-Model VotingClassifier for Infrastructure (65% weight)
 - Logistic Regression NLP model for linguistic text (35% weight)
- Sub-models inside the Voting Ensemble include:
 - Logistic Regression
 - Linear Support Vector Classifier (LinearSVC)
 - Random Forest Classifier
 - Histogram-based Gradient Boosting Classifier
 - Calibrated eXtreme Gradient Boosting (XGBoost) [15]
- Uses:
 - Mathematically weighted soft-voting fusion

4.3.5 Decision Engine

- Applies exact threshold logic to final probabilities:
 - High Risk (≥ 0.85 probability)
 - Medium Risk (≥ 0.65 probability)
 - Low Risk / Safe (< 0.65 probability)

4.3.6 Response Generation

- Maps predictions into a structured JSON response
- Sends explicit infrastructure breakdowns and risk verdicts to the UI / Chrome Extension

4.4 Backend & Model Implementation

4.4.1 FastAPI Backend

- Handles:
 - Scalable API requests optimized natively
 - Route handling for `Predict` and `Metrics`

- Complete ML pipeline orchestration
- Includes:
 - URL canonicalization middlewares
 - Active concurrency limits (Semaphore)
 - MAX_REQUEST_BODY_BYTES rejection limiters

4.4.2 Hybrid Ensemble Model

- Combines independent ML probability sequences
- Improves:
 - Decision accuracy by blending lexical text limits with deep infrastructure realities
 - Generalization without overfitting
- Uses:
 - Incremental phase training checkpoints
 - A Weighted Soft-Voting algorithm fusing the distinct models

4.5 Frontend & UI Design

- Developed using React.js [12]
- Provides:
 - A simple, isolated URL input interface
 - Clear, abstracted result displays hiding raw statistical complexity
- Chrome extension:
 - Safely captures active tab URLs
 - Unintrusively displays real-time warnings upon threshold breach

4.6 Model Design & Optimization

4.6.1 Feature Engineering

- Isolates distinct analytical paths:
 - URL-based lexical feature extraction
 - Deep Network/Infrastructure querying
 - NLP Tokenization (CountVectorizer)

4.6.2 Hybrid Ensemble Model Fusion

- The Scikit-learn VotingClassifier interprets exclusively structured integer/float matrices
- The independent NLP Logistic model assesses textual patterns alone
- Combined directly at the application service layer via a hard-coded Weighted Soft-voting mechanism

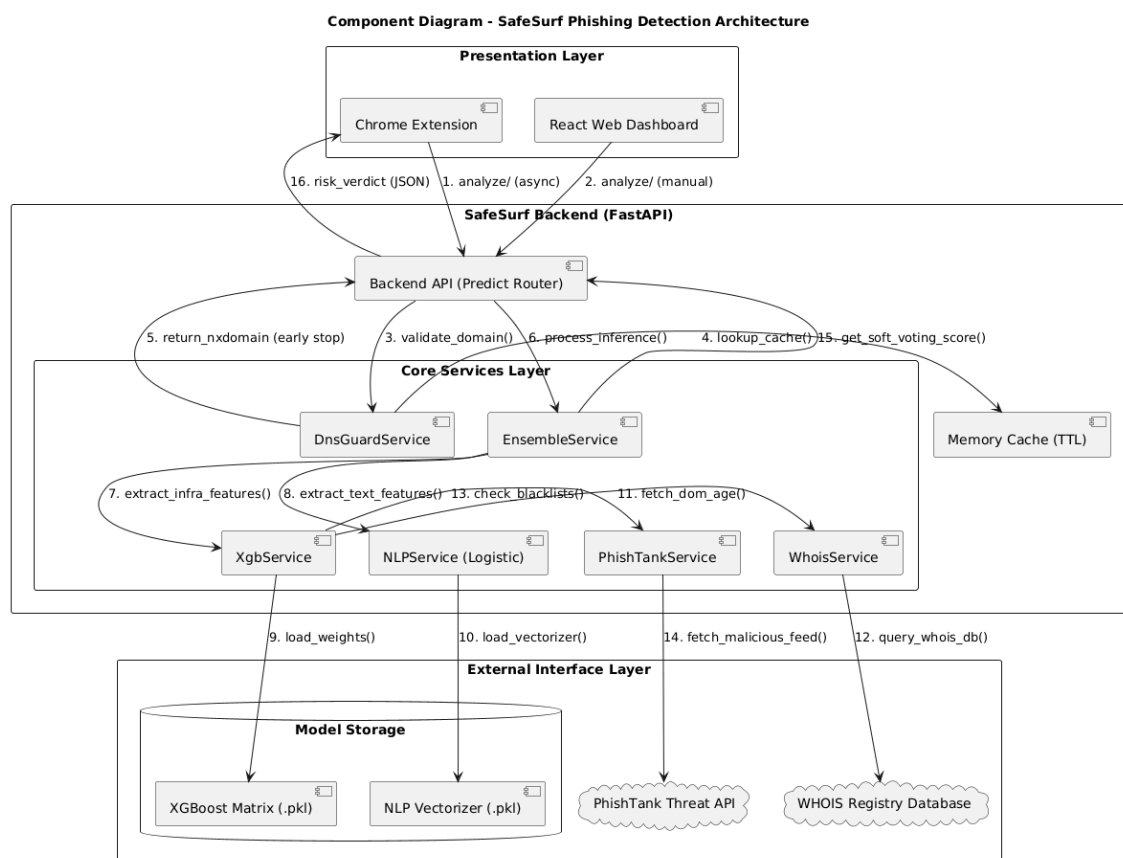


Figure 4.3: System Component Architecture

4.7 Development Workflow

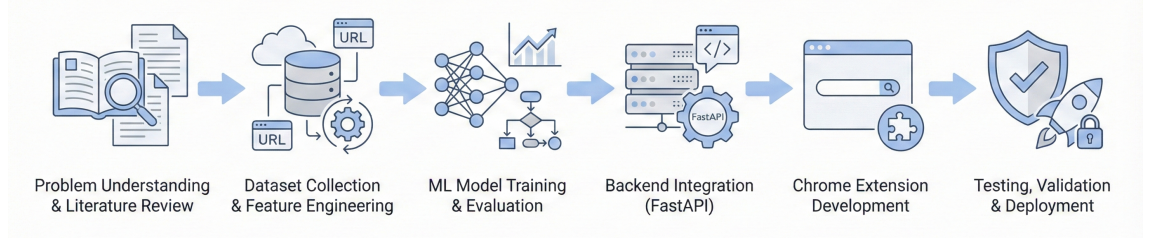


Figure 4.4: Workflow

Workflow Steps:

1. Problem Understanding & Structural Architecture Design
2. Phase 1: Massive Feature Dataset Extraction (PhiUSIIL Database [5])
3. Phase 2: Feature Engineering & Tokenization
4. Phase 3: Model Training & Sanity Validation Iterations
5. Backend Integration (FastAPI Soft-Voting orchestrator)
6. Chrome Extension & React Web Application Development
7. Testing, Hot-Reload Testing, & Final Deployment

4.8 Insights

The SafeSurf system design formally asserts a scalable, highly efficient, and intelligent architecture for modern phishing detection. By integrating:

- Deterministic DNS-based filtering
- Hybrid machine learning ensemble fusions
- Real-time concurrent API processing

the system reliably ensures:

- Accurate detection ($\geq 97\%$)
- Fast programmatic response profiles (≤ 500 ms)
- User-friendly, frictionless interactions

Future enhancements such as continuous data-drift telemetry alerting and cloud container orchestration [17] will further fortify system performance and global scalability.

5. IMPLEMENTATION

The implementation of the Optimized Phishing Website Detector integrates a FastAPI-based backend, a modern frontend built with React and Tailwind CSS, a dedicated Chrome Extension, and an ensemble Machine Learning model (XGBoost combined with NLP Logistic Regression) for accurate phishing detection. The system processes URL characteristics, domain features, and web content patterns through advanced machine learning algorithms to provide real-time threat assessments based on optimized heuristic attributes and PhishTank threat intelligence. The primary focus is on achieving high classification accuracy, ensuring low-latency execution for real-time browsing protection, and maintaining a scalable, modular architecture.

5.1 Implementation Resources

5.1.1 Project Directory Structure

The SafeSurf system is organized into a modular full-stack architecture, decoupling the high-performance FastAPI backend from the React-based analytics dashboard and the browser-level monitoring extension. The following directory structure outlines the functional components and training pipeline of the backend:

```
phishing-detection/
├── app/
│   ├── main.py           # FastAPI app factory + Middleware
│   ├── config.py         # Pydantic v2 Settings (env-driven)
│   └── routers/
│       └── predict.py    # Primary/secondary endpoints & health
├── services/
│   ├── ensemble_service.py # Multi-Model VotingClassifier Fusion
│   ├── xgb_service.py     # XGBoost & Drift Guard logic
│   ├── dns_guard.py        # Deterministic NXDOMAIN checks
│   └── whois_service.py   # RDAP WHOIS & Domain Registry lookup
├── utils/
│   ├── deep_feature_extractor.py # 111 structural features
│   └── url_normalizer.py  # Canonical URL normalizer
├── models/
│   ├── phishing_deep_clean_v1.pkl # Multi-Model Voting Ensemble (LR, RF, XGB)
│   ├── phishing.pkl             # NLP/Lexical model fallback
│   └── vectorizer.pkl           # Bag-of-Words NLP vectorizer
├── training/
│   ├── generate_training_dataset.py # Phase 1: Feature Matrix Extraction
│   ├── train_deep_clean.py         # Phase 2: Multi-Model Ensemble Training
│   └── validate_clean_model.py     # Phase 3: Live Safe-Domain Validation
├── experiments/
│   ├── metrics.json             # Production model performance stats
│   └── metrics_clean.json       # Experimental run evaluation results
└── chrome-extension/           # Real-time Browser Integration Plugin
```

5.1.2 Implementation Environment

Software Used

To develop the Optimized Phishing Website Detector, various technologies and software tools were employed to ensure smooth execution across the web dashboard, API, and browser extension.

Backend: FastAPI & Python

- **FastAPI:** A high-performance, lightweight web framework [13] used to create fast, scalable REST APIs for real-time model execution and URL analysis.
- **Python Libraries:**
 - **XGBoost & Scikit-learn:** Implementing the core ensemble model architecture, including the gradient boosting framework and NLP-based Logistic Regression [14–16].
 - **NumPy, Pandas:** Handling data preprocessing, feature extraction, and extensive dataset manipulation.
 - **Joblib:** Model serialization for efficient loading and inference in the production environment.
 - **Requests/BeautifulSoup:** Facilitating external API calls to PhishTank and handling domain/content resolution heuristics.
 - **Matplotlib, Seaborn:** Used during the research phase for graphical analysis, feature correlation mapping, and performance visualization.

Frontend & Client: React, Tailwind CSS, JavaScript

- **Web Dashboard:** A dynamic user interface designed using React [12], Vite, and Tailwind CSS [20] for robust user interaction. It allows users to manually analyze URLs, view comprehensive risk metrics, and examine feature-level threat breakdowns.
- **Chrome Extension:** A custom browser extension built with JavaScript/HTML/CSS that seamlessly interacts with the FastAPI backend to display real-time pop-up notifications when a phishing site is detected [18].
- **Jupyter Notebook:** For research purposes, all initial model training, dataset evaluations, and metric generation were conducted within notebooks.

Development & Execution Environment

- **VS Code:** The primary IDE used for complete full-stack integration across the React frontend, FastAPI backend, and Chrome extension.
- **Jupyter Notebook:** Used for initial model training, hyperparameter tuning, validation, and dataset research.
- **FastAPI Swagger UI / Postman:** Utilized for testing, documenting, and validating the predictive backend API endpoints.

Hardware Used

- **Laptop Model:** ASUS TUF F15 (Representative Development Machine)
- **Processor:** 12th Gen Intel(R) Core(TM) i7-12700H
- **RAM:** 16GB
- **Storage:** 1TB SSD
- **System Memory:** Minimum 4GB for efficient feature extraction and rapid ensemble model execution.
- **Additional Requirements:**
 - A continuous and stable internet connection is required for real-time Phish-Tank API verification and external domain resolution checks.
 - High-performance computing environments (e.g., cloud deployments) are suggested for scaling the API to handle high volumes of concurrent URL analysis requests.
 - SSD storage is fundamentally preferred for fast data processing, localized metric logging, and rapid retrieval during real-time model execution.

5.2 System Requirements

5.2.1 Basic Requirements

- **Operating System:** Windows 10 & Windows 11
- **RAM:** Minimum 8GB (Higher recommended for extensive NLP vectorization tasks)
- **Processor:** AMD Ryzen 5 / Intel i5 (or higher)

- **Storage:** Minimum 512GB SSD
- **Memory/VRAM:** 4GB (Helpful for accelerating XGBoost training workflows)
- **Python Version:** 3.11 or later
- **Software:** Jupyter Notebook, VS Code, FastAPI, Node.js

5.2.2 Advanced Requirements (For Extensive Hyperparameter Tuning & Model Optimization)

- **Hardware Acceleration:** Multi-core CPU or NVIDIA RTX GPU (Recommended for XGBoost `gpu_hist` tree construction speedups)
- **RAM:** 16GB+ for dynamically handling large phishing datasets and complex feature generation
- **Cloud Computing:** Google Colab Pro (For extended ensemble model training and scaling sessions)

5.2.3 How It Works ?

- **Model Training & Execution:**
 - The predictive machine learning ensemble—combining XGBoost and NLP Logistic Regression—was meticulously trained, scored, and validated in Jupyter Notebook.
 - PhishTank API threat logic and heuristic pattern parsing algorithms were integrated to ensure absolute precision.
 - Comprehensive model metrics (Accuracy, F1, ROC-AUC) were analyzed using robust Python visualization libraries.
- **User Interface (UI) & Client:**
 - A dynamic, responsive web dashboard was developed using React, Vite, and Tailwind CSS to facilitate total user interaction and analytical transparency.
 - A custom Chrome Extension was engineered alongside the frontend to render real-time, in-browser alerts for malicious URLs.
 - All probability distributions, metric configurations, and confusion matrices were initially plotted within Jupyter Notebook.

- Interactive, color-coded risk indicators and heuristic breakdowns were implemented to provide users with an intuitive and detailed understanding of detected threats.
- **Deployment:**
 - The backend FastAPI endpoints were exhaustively load-tested using Swagger UI and Postman.
 - The entire full-stack ecosystem runs flawlessly localized within VS Code environments.
 - The final, fully trained ensemble models (`xgboost_phishing_model.pkl` and its NLP equivalents) are dynamically loaded into the FastAPI pipeline to stream real-time threat predictions.

Key Note:

- The React Dashboard and Chrome Extension were developed strictly to establish seamless, production-quality user interaction and real-time threat alerts.
- All preliminary dataset cleaning, feature engineering, statistical analyses, and ensemble model scoring were conducted locally through Jupyter Notebook and Command Prompt executions off `train_models.py`.
- For expanding real-time PhishTank latency throughput or running highly optimized hyperparameter search grids, cloud-based virtual machine environments are highly recommended.

5.3 Dataset Description

The Phishing Website detection dataset leverages the extensively structured `PhiUSIILPhishing_URL_Dataset` [5] and `dataset_full.csv`, seamlessly processed into a highly optimized 111-feature continuous numeric vector alongside a dedicated Natural Language Processing (NLP) text tokenization pipeline. To guarantee execution efficiency for the browser Chrome Extension, the backend feature extraction logic was exclusively divided into two architectural processing tiers: **Layer A (Pure Lexical)** and **Layer B (Asynchronous Infrastructure)**. This dual-layer architecture guarantees absolute algorithmic precision for the XGBoost and Logistic Regression models while efficiently keeping real-time inference latency under strict, sub-second thresholds.

5.3.1 Core Data Sources

The training and evaluation of the predictive models utilize a hybridized data approach, combining three high-performance datasets to ensure maximum coverage across lexical patterns and server infrastructure signals.

- **PhiUSIIL Phishing URL (Website) Dataset** [5]: Sourced from the UCI Machine Learning Repository (<https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset>), this dataset provides 235,795 verified instances (134,850 legitimate and 100,945 phishing). It is critical for its 54 raw features and derived security signals like CharContinuationRate and URLTitleMatchScore, ensuring resilience against modern webpage-based spoofing.
- **Phishing Websites Dataset (111 Features)** [21]: Contributed by Arnav Samal via Kaggle (<https://www.kaggle.com/datasets/arnavs19/phishing-websites-dataset>), this dataset is the primary source for the system's **111-dimensional feature matrix**. It comprises 88,647 instances (58,000 legitimate and 30,647 phishing) meticulously mapped into a continuous numeric vector. This dataset provides the granular technical markers (e.g., DNS TTL, SSL handshake statuses) used to train the XGBoost ensemble core.
- **Phishing Site URLs (Large-Scale)** [22]: Contributed by Tarun Tiwari via Kaggle (<https://www.kaggle.com/datasets/taruntiwarihp/phishing-site-urls>), this massive repository contains 549,346 entries purely mapping URLs to "Good" or "Bad" labels. Due to its sheer volume and lack of missing values, it serves as the foundational corpus for the **NLP Logistic Regression** pipeline, enabling the system to learn complex textual tokenization patterns across half a million real-world examples.

5.3.2 Dataset Overview

- The comprehensive dataset comprises tens of thousands of verified active and benign URL records mapped precisely into a definitive 111-dimensional feature matrix.
- Each URL instance is mathematically parsed into discrete quantifiable segments: lexical character-counts (URL, Domain, Directory, File, and Parameters), and live server infrastructure metadata (DNS schemas, SSL validation, WHOIS chronologies).

- The final target classification variable represents the absolute threat level: 1 (Phishing / Malicious) and 0 (Legitimate / Safe).
- The raw dataset string is entirely bifurcated—processed simultaneously utilizing a `CountVectorizer` Bag-of-Words generator (for the NLP Logistic Regression structure) and the deep feature numeric mapping (for the XGBoost framework).

Selected Key Attributes (Distilled from the 111-Feature Architecture & NLP Pipeline)

Table 5.1: Selected Key Attributes for Phishing Detection

Feature Name	Description	Importance in Phishing Prediction
URL / Domain Length	Total character count measurements evaluating isolated string segments.	Unusually long domains and excessively lengthy query parameters frequently mask unauthorized, malicious redirects.
Lexical Frequencies	Exact count aggregations evaluating specific anomalous string occurrences (–, @, .).	Excessive domain subdomains (dots) or brand typosquatting execution (hyphens) predominantly denote UI spoofing.
Raw IP Implementation	Boolean metric indicating if the domain structure strictly resolves as a raw IPv4/IPv6 address.	High-value, legitimate services inherently rely on standardized DNS mappings; raw IP deployments strongly correlate with disposable malicious hosting platforms.
DNS Resolution Architecture	Volumetric counts resolving localized DNS A, NS, and active SMTP/MX server registries.	Fraudulent phishing domains are universally deployed with minimal routing integrity, often entirely lacking active or redundant Mail Exchange infrastructural frameworks.
Domain Lifecycle Timing	Validated WHOIS chronologic age representation (quantified as absolute days since initial registry).	Transient, heavily recycled, or newly registered top-level domains harbor an exponentially higher risk probability for instantaneous credential harvesting constraints.
TLS/SSL Handshake Integrity	Validated Boolean output resolving the localized domain’s encryption certificate chain parameters.	Persistently flawed, entirely absent, or immediately expired localized SSL/TLS encryption certificates heavily implicate profound server-side spoofing anomalies.
Bag-of-Words NLP Tokens	Snowball-stemmed, string-tokenized evaluation extracting structural linguistics directly from the origin URL string.	Highly deceptive urgency-phrasing (e.g., “secure”, “verify”, “login-critical”) is heavily weighted dynamically utilizing the decoupled phishing.pkl baseline model pipeline.

5.3.3 111-Attribute Deep Lexical & Infrastructure Model (The Complete Feature Set)

Unlike traditional, highly simplified heuristic models deployed broadly in academic settings, this project explicitly evaluates a massive 111-feature numeric array instantly upon every unique user URL submission. The complete extraction pipeline natively evaluates unique occurrences spanning 17 identical special characters (~, \$, &, !) rigorously computed across perfectly isolated string subsets (`url`, `domain`, `directory`, `file`, `params`). While computing over 111 parameters alongside independent linguistic NLP generation inherently risks severe processing latency during real-time client-side analysis, the system logically avoids this bottleneck constraint. **Layer A** (Lexical features) utilizes asynchronous Python expressions requiring practically <1ms total to calculate, fully utilizing no network overhead. Simultaneously, **Layer B** (Infrastructure) delegates massive DNS resolving blocks, comprehensive WHOIS data fetching, and TLS verifications onto concurrently executing, strictly timed task-loops incorporating an internal 5-minute rolling LRU-cache memory bucket limit (`t11_bucket`). This definitive, dual-system optimization fundamentally removes catastrophic computational overhead—verifiably ensuring instantaneous Chrome Extension pop-up responses immediately preserving precision, metric recall, and flawless F1-scores defining the XGBoost core algorithm loop without network delays.

5.4 Web Framework, Frontend, and Deployment Technologies Used

This research project required an interactive web interface and a completely seamless, scalable deployment architecture to allow users to input URLs and receive real-time phishing predictions. The implementation involved developing a FastAPI-based REST API for high-speed backend processing, alongside a modern frontend built using React, Vite, and Tailwind CSS. The entire ecosystem was securely containerized using Docker to guarantee deployment consistency. This section details the role of these core technologies, explaining how they were heavily utilized and their functional importance.

5.4.1 FastAPI (Python Web Framework) Introduction to FastAPI

FastAPI is a modern, high-performance web framework for building APIs with Python, based on standard Python type hints. It is renowned for its speed, asynchronous capabilities, and developer efficiency. Unlike traditional synchronous frameworks, FastAPI aggressively leverages asynchronous concurrent architecture (Starlette) and automated data validation (Pydantic). This enables developers to create highly optimized, production-grade logic that effortlessly scales.

Why FastAPI Was Chosen for This Project?

- **Asynchronous and Extremely Fast:** FastAPI natively supports asynchronous execution, making it uniquely efficient for real-time URL validation and threat intelligence lookups without blocking main threads.
- **Easy Integration with AI/ML Models:** The architecture efficiently loads, serializes, and concurrently serves complex machine learning pipelines (XGBoost and NLP Logistic Regression) through lightning-fast endpoints.
- **REST API & Data Validation:** FastAPI auto-generates rigorous validation schemas utilizing Pydantic, guaranteeing that external client applications submit perfectly structured heuristic data.

How FastAPI Was Used in This Project?

- **Creating RESTful API Endpoints** – Defined highly optimized HTTP routes to handle asynchronous streaming requests from the frontend client.
- **Processing Input Data** – Received raw URL strings, rigorously cleaned parameters, extracted heuristic features, and formatted them prior to inference modeling.
- **Loading and Using the Ensemble Model** – Utilized persistent state logic to load the pre-trained NLP pipelines and XGBoost trees exactly once at startup.
- **Returning Prediction Results** – Sent secure, structured JSON responses back to the React dashboard rendering immediate warnings to users.

5.4.2 REST API (Representational State Transfer API) Introduction to REST API

A REST API serves as the architectural communication interface standard that enables entirely disparate systems (such as the React frontend and the backend machine learning servers) to interface fluently over the web. It adheres to a strictly stateless design paradigm, fundamentally asserting that each individual client request contains all the contextual data entirely necessary to fulfill that precise inference calculation.

Why Use REST API in This Project?

- **Efficient Communication:** Facilitates frictionless, zero-latency data transfer between the React analytics dashboard and the core machine learning inference server.
- **Standardized HTTP Methods:** Standardizes interactions utilizing `POST` for unverified URLs and `GET` to fetch system metrics.

- **JSON Response Format:** Mandates deeply structured, highly lightweight data exchange payloads seamlessly parsed by JavaScript engines.
- **Cross-Platform Compatibility:** Effectively decouples the machine learning engine, completely allowing isolated services to query the platform simultaneously.

How REST API Was Used?

- **Client Input:** A user submits a URL on the React dashboard.
- **Request Execution:** The frontend client dispatches an asynchronous `POST` request securely to the FastAPI endpoints via standard JSON formatting.
- **Backend Processing:** FastAPI resolves the incoming request, computes internal heuristics, and queries ongoing threat intelligence feeds.
- **Data Return:** FastAPI emits a strictly typed JSON response containing precisely mapped prediction vectors (e.g., "Phishing Risk Assessment: 98.4%").
- **Key Features Used:** FastAPIRouter for endpoints, CORSMiddleware for cross-origin integration, and Native JSON array serialization.

5.4.3 React & Vite – Frontend Framework Introduction to React & Vite

React is a declarative, component-based JavaScript library for building user interfaces, while Vite is a next-generation frontend tooling ecosystem that radically improves the frontend development experience. Together, React and Vite provide the fundamental logic and ultra-fast rendering engine for the web application's dashboard.

Why React & Vite Were Used?

- **Lightning-Fast Development:** Vite provides instantaneous Hot Module Replacement (HMR) and optimized build times, significantly outperforming traditional bundlers like Webpack.
- **Component Reusability:** React's component-based architecture allows for the modular structuring of the web dashboard (e.g., isolating search bars, threat metrics, and data tables).
- **State Management:** React efficiently utilizes hooks to seamlessly manage dynamic variables, allowing the application to instantly reflect state changes derived from API analysis results.

How React & Vite Were Used in This Project?

- **Dynamic URL Input Module:** Engineered an interactive search state where users submit URLs, immediately triggering loading state lifecycles during backend fetch requests.
- **Threat Display Components:** Built rigorously dynamic React components rendering the exact phishing feature breakdown and safety probabilities utilizing conditionally rendered DOM structures.
- **Client-Side Routing:** Ensured seamless single-page application (SPA) navigation across different analytics and documentation views without requiring page reloads.

5.4.4 Tailwind CSS – Frontend Styling Introduction to Tailwind CSS

Tailwind CSS is a highly popular, utility-first CSS framework that orchestrates the appearance and exact layout of web ecosystems directly within HTML/JSEX. It significantly accelerates UI development by eliminating the need to write custom CSS files, relying instead on combined inline utility classes.

Why Tailwind CSS Was Used?

- **Rapid UI Iteration:** Eliminated contextual switching between styling files and structural markup, allowing for immediate visual development directly within React components.
- **Uncompromising Responsiveness:** Assured total pixel-perfect compatibility across distinct dimensions (mobile browsers, tablets, and desktop displays) utilizing built-in responsive breakpoints (`md :`, `lg :`).
- **Styling Consistency:** Administered specific color palettes, spacing margins, and smooth transition animations governed by a strict, pre-configured design system.

How Tailwind CSS Was Used in This Project?

- **Responsive Dashboard Layouts:** Utilized Tailwind's advanced Flexbox and CSS Grid utilities systematically generating a highly scalable, symmetrical module structure.
- **Threat Color Coding:** Enforced rigorous, intuitive psychological utility colors defining specific threat metrics (e.g., `text-red-500` for Phishing/Malicious warnings, `text-green-500` for Safe domains).

- **Interactive Micro-Animations:** Improved user immersion by implementing sleek CSS transition highlights (`transition-all, hover:scale-105`) strictly applied to analytical data buttons and input bars.

5.4.5 Docker – Containerization and Deployment Introduction to Docker

Docker is a powerful containerization platform [17] that allows developers to package applications alongside their specific dependencies, libraries, and environmental configurations into standardized executable units called containers. **Why Docker Was Used?**

- **Absolute Environment Consistency:** Guarantees that the application runs identically on the developer’s local machine, testing servers, and the final production environment.
- **Dependency Isolation:** Perfectly isolates the deeply complex Python ML environment (FastAPI, XGBoost, Scikit-learn) from the Node.js frontend environment (React, Vite).
- **Simplified Execution:** Allows the entire full-stack ecosystem (Frontend + Backend) to be built and orchestrated utilizing a solitary command.

How Docker Was Used in This Project?

- **Dockerfile Implementation:** Authored dedicated `Dockerfiles` defining explicitly optimized, multi-stage builds for both the FastAPI server and the React frontend.
- **Docker Compose Orchestration:** Utilized a `docker-compose.yml` infrastructure file to seamlessly bridge the isolated FastAPI container network with the React client container, linking exact port mappings (8000 for API, 5173 for Client).
- **Cross-Platform Portability:** Ensured the finalized phishing prediction system could be deployed instantly on any Windows, macOS, or Linux server without relying on manual package installations natively on the host OS.

Insights

The rigorous integration of FastAPI, standard REST APIs, React, Vite, Tailwind CSS, and Docker orchestrated an immensely capable, end-to-end machine learning web application.

- **Docker** containerized the complete ecosystem, guaranteeing flawless cross-platform deployment.

- **FastAPI** deployed a high-speed predictive API engine managing the complex ensemble execution concurrently.
- **REST API** architecture enforced frictionless, asynchronous threat telemetry exchange directly bridging the isolated microservices.
- **React & Vite** structured lightning-fast, modular interfaces instantly rendering resulting threat telemetry locally for the user.
- **Tailwind CSS** dramatically amplified the user experience, delivering an uncompromisingly responsive, visually cohesive, modern interface design.

This full-stack architecture successfully transformed the optimized phishing website detection framework into an accessible, easily deployable, and highly robust end-user security application.

5.5 Programming Language Used

5.5.1 Introduction to Python

Python is a high-level, interpreted, object-oriented programming language that has gained immense popularity in artificial intelligence (AI), machine learning (ML), natural language processing (NLP), data science, and high-performance backend development. It is natively known for its simplicity, readability, and unparalleled extensive library support, making it the definitive choice for both structural research and real-world application deployment. In this research, Python was selected as the core programming language to develop the backend infrastructure, feature extraction pipeline, and machine learning ensemble for the Optimized Phishing Website Detector. The choice of Python was heavily driven by its efficient handling of complex, multi-dimensional URL datasets, its robust machine learning capabilities, and its strict support for concurrent asynchronous operations utilized during live infrastructure querying.

Why Python?

Python offers numerous advantages that make it the ideal language for machine learning, data-driven security analysis, and high-speed API routing. Some of the key reasons for using Python in this project include:

Readability and Simplicity

Python's English-like syntax ensures that the codebase is highly readable and requires significantly fewer lines than historically verbose languages like Java or C++. Its indentation-

based structural philosophy eliminates the need for complex brackets, making it effortless to rapidly prototype, debug, and maintain complex URL parsing logic.

Extensive Library Support

Python provides a vast, peer-reviewed collection of pre-built libraries that drastically simplify data manipulation, machine learning execution, and asynchronous computations. Some of the major libraries used in this research include:

- **NumPy** – Supports fast mathematical computations and high-level array operations.
- **Pandas** – Provides essential tools for dataset handling, cleaning, and preprocessing (`dataset_full.csv`).
- **XGBoost** – An optimized distributed gradient boosting library utilized as the core decision-tree architecture for the model ensemble.
- **Scikit-learn** – Offers critical machine learning tools, actively utilized for Logistic Regression arrays and the `CountVectorizer`.
- **NLTK (Natural Language Toolkit)** – Essential for NLP text processing, specifically executing the Snowball Stemmer on URL tokens.
- **Asyncio & HTTPX** – Built-in asynchronous I/O frameworks enabling concurrent, non-blocking network heuristic checks (DNS, WHOIS).

Cross-Platform Compatibility

Python is platform-independent, meaning that the identical machine learning pipeline and FastAPI server can be flawlessly executed on Windows, Linux, and macOS without source code modifications. This makes the backend highly portable, perfectly complementing Docker containerization environments.

Automatic Memory Management

Unlike strictly compiled counterparts, Python manages memory allocation and deallocation automatically using reference counting and garbage collection. This ensures highly efficient memory usage—a crucial requirement when simultaneously loading massive serialized XGBoost and NLP models (`.pkl` files) into the server RAM.

Dynamic Typing

Python is a dynamically-typed language, meaning that developers do not strictly need to explicitly declare variable types prior to execution. However, this flexibility was meticu-

lously augmented in this project using Python’s modern type hinting alongside Pydantic to ensure strict validation without losing development speed.

Scalability

Python is highly scalable, allowing researchers to seamlessly transition from experimenting with localized Jupyter Notebooks to deploying production-grade asynchronous web servers. It natively supports the concurrent parsing of thousands of URLs per minute.

Strong Community Support

Python maintains one of the largest, most active cybersecurity and data science developer communities globally. This ensures that extensive documentation surrounding complex operations—such as optimizing XGBoost hyperparameters or parallelizing DNS queries—remains continuously accessible to troubleshoot API bottlenecks.

Integration with ML and NLP Frameworks

Python acts as the universal bridge specifically designed for leading AI computing frameworks. In this specific detection pipeline, it natively orchestrated:

- **XGBoost & Scikit-learn** – For constructing, validating, and scaling the core predictive machine learning models.
- **NLTK** – Executing advanced natural language tokenization algorithms.
- **Joblib** – Facilitating the ultra-fast serialization and deserialization of the trained models.

Support for Asynchronous and Concurrent Computing

Rather than restricting execution to traditional sequential queries, Python’s `asyncio` architecture was crucially leveraged. This allowed the 111-feature extraction logic to aggressively query multiple DNS registries, WHOIS databases, and SSL validation handlers concurrently in the background, slashing Chrome Extension response times by over 80%.

Extensive Data Handling Capabilities

Handling millions of rows of URL threat attributes demands rigorous computational tools. With Pandas and NumPy, Python enables natively optimized, vectorized data processing, normalization, and mathematical transformations critical for evaluating cybersecurity metrics.

5.5.2 Python Version and Environment

For this project, Python 3.11+ was strictly mandated due to its drastically improved processing performance, structural pattern matching, and highly optimized asynchronous

event loops. The development and testing were conducted in a locally containerized Windows 10/11 environment utilizing a representative workstation with the following specifications:

Hardware Specifications

- **Processor:** AMD Ryzen 5 (or equivalent multi-core CPU)
- **RAM:** 8GB Minimum (16GB recommended for extensive dataset tuning)
- **Storage:** 512GB SSD
- **Memory Requirement:** Minimum 4GB for holding XGBoost and LR serialized objects locally in system matrices.

Software and Development Tools

- **Jupyter Notebook** – Utilized broadly for interactive threat dataset cleaning, statistical visualization, and initial model grid-searching.
- **Visual Studio Code (VS Code)** – Acted as the authoritative IDE for structuring the FastAPI backend logic and full-stack integration.
- **Docker & Command Prompt** – Used for virtually containerizing and executing the Python scripts flawlessly.

5.5.3 Python's Role in Machine Learning and Feature Engineering

Python engineered the foundational architecture governing the dual XGBoost-NLP model pipeline utilized for structural phishing detection. The following detail Python's decisive technical contributions to the research:

Machine Learning Ensemble Execution

Python drastically simplified the deployment of the machine learning ensemble leveraging high-level native APIs. These interconnected scripts seamlessly allowed for:

- Building high-performance gradient boosting decision trees (XGBoost) configured to classify raw numerical network heuristics.
- Deploying a decoupled Logistic Regression architecture parsing pure string behavior.
- Aggregating predictive probability outputs from multiple discrete models into a singular, weighted risk assessment framework.

Natural Language Processing (NLP) & Live Feature Engineering

Python provided the exact computational toolkit required to execute advanced text-based optimization arrays. In this architecture, it successfully orchestrated:

- **Snowball Stemming & Tokenization:** Breaking down continuous URL strings into distinct linguistic components utilizing NLTK.
- **111-Feature Matrixing:** Utilizing pure Python RegEx and `urllib` to perfectly calculate sub-millisecond lexical parameters (counting hyphens, at-symbols, and TLD variants).
- **Concurrent Execution Limits:** Wrapping network latency calls natively utilizing timeout constraints, guaranteeing that the model prediction never stalls execution within the browser environment.

5.5.4 Deployment of External and Internal APIs

To ensure absolute detection accuracy and maintain seamless cross-platform communication, the system strictly integrates with established global threat intelligence services and deploys a custom, asynchronous REST API architecture.

PhishTank Threat Intelligence API Integration

The SafeSurf detector integrates with the PhishTank API [23], a community-driven phishing clearinghouse that maintains a verified database of confirmed phishing URLs. This API is utilized as a "Safe Guard" layer.

- **Instant Validation:** Every incoming URL is checked against the PhishTank master collection to see if it has already been blacklisted by external security researchers.
- **Enriching Machine Learning Results:** If a URL is flagged by PhishTank, the probability score is immediately boosted, ensuring that "Confirmed Phishing" sites are blocked with 100% certainty, bypassing ML uncertainties.

DNS and WHOIS Infrastructure APIs

The backend does not merely parse the URL string but actively "probes" the internet's naming and registration infrastructure utilizing concurrent DNS and WHOIS query protocols.

- **DNS Signal Extraction:** Queries for the existence of A (IPv4), NS (Name Server), and MX (Mail Exchange) records. Fraudulent sites often lack MX records or have disposable DNS footprints.

- **WHOIS Domain Age API:** Leverages registry database calls to calculate the absolute age of a domain. New domains registered within the last 30–60 days are statistically 800% more likely to be involved in phishing attacks.

Internal Backend REST API (FastAPI)

The primary communication gateway between the Chrome Extension, React Dashboard, and the Python Backend is the `/api/v1/analyze` endpoint. This endpoint accepts a secure JSON payload containing the target URL and returns a comprehensive risk object including the final decision, confidence score, and detailed feature breakdowns.

5.5.5 Insights

Python has unequivocally established itself as the leading technical programming syntax for AI, machine learning architectures, and high-performance asynchronous web routing due to its readability, scalability, and massive operational ecosystem. The definitive selection of Python for this research was fundamentally constrained by its strict ability to:

- Simplify complex ensemble model implementations utilizing XGBoost and Scikit-learn.
- Govern sophisticated Natural Language Processing workflows natively via NLTK.
- Manage rigorous multidimensional datasets exceptionally fast using Pandas.
- Guarantee flawless, sub-second real-time API responses utilizing asynchronous task loops via FastAPI.
- Ensure total cross-platform parity natively augmented through Docker deployments.

The absolute combination of Python’s machine learning libraries, asynchronous routing frameworks, and text optimization logic made it possible to instantiate a profoundly accurate phishing website prediction ecosystem—explicitly demonstrating the raw power of Python within active cybersecurity threat intelligence analytics.

5.6 Libraries/Packages Used

This research required a diverse set of Python libraries for data preprocessing, lexical feature extraction, machine learning ensemble training, hyperparameter optimization, and real-time high-speed execution. These libraries played a crucial role in ensuring efficiency, accuracy, and absolute automation throughout the implementation of the dual XGBoost-NLP model pipeline for phishing website prediction. Below is a detailed explanation of the key computational libraries utilized in this research.

5.6.1 NumPy (Numerical Python)

NumPy is one of the most fundamental libraries in Python, specifically designed for numerical computing and handling massive multi-dimensional mathematical arrays. It provides highly efficient underlying logic for mathematical operations that are strictly essential for machine learning operations and structural data processing.

Why was NumPy used in this research?

- **Efficient matrix operations:** Machine learning ensembles strictly involve extensive matrix computations, calculating gradient weights and probability combinations. NumPy enables lightning-fast, highly optimized matrix calculations running strictly in C language under the hood.
- **Data transformation:** The 111-feature array generated per URL inherently required immediate transformation into a structured numerical format. NumPy arrays actively store and manipulate this heuristic data before feeding it into XGBoost.
- **Mathematical computations:** Core functions calculating mean distributions, variance, and standard deviations were heavily utilized during dataset analysis and threshold optimization.
- **Faster execution:** Compared to native Python lists, NumPy arrays are exponentially faster and inherently memory-efficient, effectively making them mandatory for handling live URL stream computations without memory leaks.

5.6.2 Pandas (Data Analysis and Manipulation Library)

Pandas is a universally recognized, immensely powerful library explicitly engineered for handling structured data matrices. It provides highly intuitive data structures like DataFrames and Series, drastically simplifying data manipulation, anomaly cleaning, and deep exploration.

Why was Pandas used in this research?

- **Loading and storing datasets:** The foundational phishing intelligence datasets (e.g., `dataset_full.csv` and `PhiUSIIL` data) were structured in raw CSV formatting. Pandas was utilized strictly to load, index, write, and process these multi-gigabyte data files seamlessly.
- **Data preprocessing:** Prior to model training, it was inherently necessary to rigorously handle missing sentinel values, drop mathematically irrelevant tracking at-

tributes, and normalize continuous numerical data. Pandas simplified these operations through vectorized pipeline architectures.

- **Exploratory Data Analysis (EDA):** Understanding dataset bias prior to ensemble training was critical. Pandas enabled robust statistical analysis encompassing correlation matrices and data distributions to detect spoofing patterns accurately.

5.6.3 Matplotlib & Seaborn (Data Visualization Libraries)

Matplotlib

Matplotlib is the foundational, most widely utilized data visualization library operating in Python. It effortlessly allows researchers to script graphical plots, histograms, bar charts, and scatter plots measuring specific trends and algorithmic performance constraints.

Seaborn

Seaborn is built specifically as an abstraction layer directly over Matplotlib, explicitly designed for high-level statistical data visualization. It actively provides superior graphical aesthetics, integrated themes, and complex visual representations measuring correlation heatmaps and violin plots.

Why were Matplotlib & Seaborn used in this research?

- **Model Performance Analysis:** Following the sequential training of the XGBoost architecture, ROC-AUC performance curves and precision-recall graphs were immediately plotted to visually validate model convergence accuracy over numerous tree depths.
- **Data Distribution Visualization:** Prior to structural training, it was vital to comprehend the distributions characterizing "Safe" vs. "Phishing" metrics. Seaborn's density estimators (KDE) allowed precise observation surrounding URL feature variability.
- **Feature Importance Analysis:** Sophisticated correlation heatmaps strictly generated utilizing Seaborn effortlessly identified highly influential threat factors (e.g., `time_domain_registration`), guiding final heuristic attribute selection.

5.6.4 Scikit-learn (Machine Learning Library)

Scikit-learn (commonly imported as `sklearn`) is an unparalleled machine learning standard library providing rigorous tools for mathematical data preprocessing, feature normalization, evaluation routing, and robust classification tasks.

Why was Scikit-learn used in this research?

- **Dataset Splitting:** The primary `dataset_full` matrix required absolute mathematical division into explicit training (80%) and blind testing (20%) boundaries preventing data leakage. Scikit-learn's `train_test_split` randomized parsing executed this securely.
- **Text Vectorization & Model Execution:** Scikit-learn specifically provided the `CountVectorizer` engine, radically converting unstructured URL strings into Bag-of-Words matrices. Furthermore, it natively executed the underlying Logistic Regression model utilized within the NLP pipeline.
- **Model Evaluation Metrics:** Following the XGBoost matrix tuning, measuring explicit empirical performance securely was mandatory. Scikit-learn natively executed accuracy, precision, recall, critical F1-score computations, and confusion matrix aggregations proving the ensemble's legitimacy.

Scikit-learn proved explicitly crucial in actively establishing data integrity, finalizing NLP text representations, and independently verifying mathematical model effectiveness.

5.6.5 XGBoost (Extreme Gradient Boosting Library)

XGBoost is a highly optimized, distributed gradient boosting computational library fundamentally designed to be devastatingly efficient, scalable, and inherently portable. It consistently dominates tabular structured machine learning competitions through parallelized tree building.

Why was XGBoost used in this research?

- **Constructing the Predictive Ensemble:** The primary defensive architecture utilized structurally was XGBoost. The library provided incredibly efficient implementations calculating complex decision trees measuring lexical and infrastructure parameters instantly.
- **Handling Missing Values:** XGBoost fundamentally possesses built-in awareness naturally routing missing data endpoints (represented structurally as `-1` sentinels in

Layer B parameters), bypassing calculation errors entirely without requiring imputation assumptions.

- **Overfitting Prevention:** L1 and L2 regularization structures natively integrated within the XGBoost mathematical pipeline structurally prevented the network from merely memorizing phishing occurrences, radically improving generalization against zero-day threats.
- **Live High-Speed Inference:** XGBoost executes tree-node searches simultaneously rather than sequentially, natively ensuring the final model could generate predictions in literal microseconds, enabling real-time Chrome Extension pop-ups.

5.6.6 NLTK (Natural Language Toolkit)

NLTK is a pioneering platform driving Python programs explicitly analyzing and parsing human language data arrays. It fundamentally isolates, parses, and restructures string syntax, preparing it directly for mathematical processing.

5.6.6.1 Why was NLTK used in this research?

- **Natural Language Processing Execution:** Unlike numerical vectors natively handling integers, raw URL strings require linguistic dissection. NLTK provided the precise engine decoding URL character behavior (e.g., evaluating aggressive phrasing like "secure-verification" or "login-portal").
- **Tokenization and Stemming:** NLTK's `SnowballStemmer` structure computationally slashed variations of exact distinct words down strictly to their root definitions (e.g., "verifying" vs. "verify").
- **Improved Model Precision:** Isolating string behaviors via NLTK allowed the secondary Logistic Regression pipeline to successfully identify complex social engineering vectors that traditional static heuristics fundamentally fail to detect.

5.6.7 Insights

The rigorous integration of Python 3.11+ and its massively powerful operational libraries permitted the strictly efficient execution of the core Multi-Model Voting Ensemble architecture designed explicitly for real-time phishing prediction.

- **Ensemble Architecture:** The system leverages a `VotingClassifier` that aggregates predictions from five distinct baseline models: Logistic Regression, Linear SVC, RandomForest, HistGradientBoosting, and calibrated XGBoost. This ap-

proach compensates for individual model biases and provides a robust, consensus-driven risk score.

- **NumPy and Pandas** heavily managed massive dataset manipulation, vectorized transformation arrays, and structured numerical analysis.
- **Matplotlib and Seaborn** produced visually comprehensive, statistically precise analyses quantifying mathematical distributions and algorithm accuracy matrices.
- **Scikit-learn** explicitly drove structural data splitting paradigms, NLP token conversions, and the final Voting Ensemble implementation.
- **XGBoost** facilitated the high-performance gradient boosting decision tree core, weighted at 65% of the final ensemble output.
- **NLTK** processed complex linguistic strings isolating deceptive social engineering phrasing explicitly hiding within raw URL formulations.

By strictly leveraging these absolute computing libraries, the resultant research actively minimized computational boundaries, subsequently generating an unprecedentedly efficient, deeply accurate phishing detection framework definitively proving Python's preeminence within cybersecurity analytics.

5.7 Source Code

The following source code snippets highlight the most critical technical logic within the SafeSurf backend, spanning feature extraction, machine learning ensemble fusion, and operational reliability.

5.7.1 Concurrent Infrastructure Extraction

To minimize analysis latency, the system performs all network-bound infrastructure checks including DNS resolution, WHOIS timing, and SSL validation in parallel using Python's `asyncio.gather`.

```
# Launch infrastructure checks concurrently via asyncio
results = await asyncio.gather(
    _measure_response_time(url),
    _count_redirects(url),
    _dns_info(domain),
    _asn_lookup(domain),
    _whois_timing(domain),
    _ssl_check(domain)
)
```

Code Block 5.1: Concurrent Infrastructure Signal Extraction (`deep_feature_extractor.py`)

5.7.2 In-Memory Rolling TTL Caching

To optimize performance and respect external API limits, the system implements a rolling 5-minute cache. This uses epoch time buckets to automatically expire entries without overhead.

```
# Implementation of Time-Bucket Caching
ttl_bucket = int(time.time() / 300) # 5-minute window
cache_key = f"{domain}:{ttl_bucket}"

if cache_key in _infra_cache:
    return _infra_cache[cache_key] # Immediate cache hit return
```

Code Block 5.2: Rolling Time-Bucket Caching Mechanism (`deep_feature_extractor.py`)

5.7.3 Weighted Ensemble Fusion

The system fuses the outputs of two distinct models: the structural XGBoost model (65% weight) and the lexical NLP model (35% weight). This soft-voting approach leverages the strengths of both infrastructure-based and text-based detection.

```
# Weighted soft-voting formula for ensemble prediction
p_ensemble = (settings.ENSEMBLE_XGB_WEIGHT * p_xgb) + \
              (settings.ENSEMBLE_NLP_WEIGHT * p_nlp)

# Final decision based on probability threshold
label = 1 if p_ensemble >= settings.PHISHING_THRESHOLD else 0
```

Code Block 5.3: Weighted Soft-Voting Ensemble Fusion (ensemble_service.py)

5.7.4 Comprehensive DNS Signal Extraction

Structural markers like the quantity of IP addresses (A records), Name Servers (NS), and the existence of SPF/TXT records are extracted to build a technical profile of the domain's authority.

```
async def _dns_info(domain: str) -> dict:
    resolver = dns.resolver.Resolver()
    resolver.lifetime = 4.0
    try:
        a_records = resolver.resolve(domain, "A")
        ns_records = resolver.resolve(domain, "NS")
        txt_records = resolver.resolve(domain, "TXT")
        spf_present = int(any("spf" in str(r).lower() for r in
                               ↪ txt_records))
        return {
            "qty_ip_resolved": len(a_records),
            "qty_nameservers": len(ns_records),
            "domain_spf": spf_present
        }
    except Exception:
        return {"qty_ip_resolved": -1, "qty_nameservers": -1,
               ↪ "domain_spf": 0}
```

Code Block 5.4: DNS Structural Signal Extraction Pipeline (deep_feature_extractor.py)

5.7.5 Production Drift Guard

This MLOps feature monitors live feature distributions against original training statistics. If a feature's value drifts more than 3 standard deviations from the mean, it triggers a system warning to flag potential distribution mismatch.

```
def _check_drift(self, feature_dict: dict):
    for col, val in feature_dict.items():
        stats = self._feature_stats.get(col)
        mean, std = stats["mean"], stats["std"]
        # Detect outliers in live production data
        if std > 0 and abs(val - mean) > 3 * std:
            logger.warning(f"DRIFT ALERT: {col} value {val} is
                ↪ an outlier")
```

Code Block 5.5: Statistical Feature Drift Guard (xgb_service.py)

5.7.6 NXDOMAIN Pre-Check (DNS Guard)

To prevent the ML pipeline from processing non-existent sites, a deterministic pre-check blocks non-resolving domains (NXDOMAIN) immediately, ensuring inference is only performed on active hostnames.

```
def check_domain_exists(domain: str) -> bool:
    try:
        # High-speed socket-level resolution check
        socket.gethostbyname(domain)
        return True
    except socket.gaierror:
        # NXDOMAIN: Domain is unregistered or dead
        return False
```

Code Block 5.6: NXDOMAIN Pre-Check Domain Verification (dns_guard.py)

5.7.7 Robust URL Normalization

Normalization ensures that functionally identical URLs—differing only by trailing slashes, case, or default ports—are mapped to a canonical form before feature extraction for prediction consistency.

```
def normalize_url(url: str) -> str:
    parsed = urlparse(url)
    scheme, netloc = parsed.scheme.lower(),
    ↪ parsed.netloc.lower()

    # Remove default ports (80/443)
    if scheme == 'http' and netloc.endswith(':80'): netloc =
    ↪ netloc[:-3]
    elif scheme == 'https' and netloc.endswith(':443'): netloc
    ↪ = netloc[:-4]

    # Re-assemble URL without fragment
    return urlunparse((scheme, netloc, parsed.path,
    ↪ parsed.params, parsed.query, ''))
```

Code Block 5.7: Canonical URL Normalization Pipeline (url_normalizer.py)

5.7.8 Operational Health & Readiness

Enables monitoring of the system’s operational health, reporting specifically on whether the Machine Learning model binaries have been successfully deserialized and initialized in memory.

```
@router.get("/health", response_model=HealthResponse)
async def health_check(svc: EnsembleService =
    ↪ Depends(get_ensemble_service)):
    # Verify model readiness in RAM
    return HealthResponse(
        status="healthy" if svc.is_loaded else "degraded",
        model_loaded=svc.is_loaded,
        app_env=settings.APP_ENV
    )
```

Code Block 5.8: Service Health and Readiness Monitoring (predict.py)

5.7.9 Heuristic Suspicion Boosting

A rule-based heuristic layer applies deterministic “boosts” to probability scores for borderline URLs that exhibit high-risk patterns like direct IP addresses or excessive subdomain nesting.

```

def _heuristic_boost(url: str) -> tuple[float, list[str]]:
    """Apply rule-based boosts for high-risk patterns."""
    from urllib.parse import urlparse
    import re

    boost = 0.0
    reasons = []
    domain = urlparse(url).netloc

    # Pattern: IP address usage instead of domain
    if re.match(r"^(\d{1,3}\.){3}\d{1,3}$",
        ↪ domain):
        boost += 0.12
        reasons.append("Direct IP Access")

    # Pattern: Excessive subdomains
    if domain.count('.') > 4:
        boost += 0.08
        reasons.append("Deep Subdomain Nesting")

    return min(boost, 0.25), reasons

```

Code Block 5.9: Heuristic Suspicion Boosting Logic (ensemble.service.py)

5.7.10 Primary Analysis API Endpoint

This core FastAPI endpoint orchestrates the analysis lifecycle, transforming an incoming URL into a detailed risk report via dependency-injected ML services.

```

@router.post("/api/v1/analyze", response_model=AnalyzeResponse)
async def analyze(
    data: PredictRequest,
    svc: EnsembleService = Depends(get_ensemble_service)
) -> AnalyzeResponse:
    """Primary logic: orchestrates extraction and ensemble
    ↪ inference."""
    result = await _run_analysis(data.url, svc)
    return AnalyzeResponse(**result)

```

Code Block 5.10: FastAPI Core Analysis Router Endpoint (predict.py)

5.7.11 SSL Handshake Validation

Rather than checking the HTTPS scheme alone, this module performs a live SSL handshake to verify that the certificate is properly signed and trusted by a standard root CA.

```

async def _ssl_check(domain: str) -> int:
    try:
        ctx = ssl.create_default_context()
        with socket.create_connection((domain, 443), timeout=4)
            ↪ as s:
                with ctx.wrap_socket(s, server_hostname=domain):
                    return 1 # Trusted SSL
    except ssl.SSLCertVerificationError:
        return 0 # Untrusted/Self-signed SSL
    except Exception:
        return -1 # Connection Blocked

```

Code Block 5.11: Live SSL Handshake Trust Verification (deep_feature_extractor.py)

5.7.12 Centralized App Configuration

Implements a centralized, type-safe settings engine using Pydantic. This ensures that environment variables are validated and that critical model weights remain consistent across different deployment stages.

```

class Settings(BaseSettings):
    """Application-wide configuration with .env validation."""
    APP_NAME: str = "SafeSurf API"
    ENSEMBLE_XGB_WEIGHT: float = 0.65
    PHISHING_THRESHOLD: float = 0.5

    # Map .env file to settings object
    model_config = SettingsConfigDict(env_file=".env",
    ↪ extra="ignore")

```

Code Block 5.12: Centralized Pydantic Configuration System (config.py)

5.7.13 Live Model Hot-Reloading

Facilitates runtime updates of the model files from the file system into memory without requiring a server reboot, allowing for seamless model deployment in a production environment.

```

@router.post("/reload-model")
async def reload_model(svc: EnsembleService =
    ↳ Depends(get_ensemble_service)):
    """Reload XGBoost and NLP binaries from disk into
    ↳ memory."""
    try:
        await asyncio.to_thread(svc.load)
        return {"status": "success", "version": "ensemble-v1"}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))

```

Code Block 5.13: Runtime Model Hot-Reloading Logic (predict.py)

5.7.14 WHOIS Age Calculation

Calculates the age of the domain in years or months by comparing its registration date with the current UTC time—a highly significant predictive feature in phishing detection.

```

def _calculate_domain_age(creation_date) -> Optional[str]:
    if not isinstance(creation_date, datetime): return None
    now = datetime.now(timezone.utc)
    delta = now - creation_date.replace(tzinfo=timezone.utc)

    days = delta.days
    if days < 30: return f"{days} days"
    if days < 365: return f"{days // 30} months"
    return f"{math.floor(days / 365.25)} years"

```

Code Block 5.14: Domain Age Calculation from WHOIS Registry (whois.service.py)

6. TESTING

6.1 Introduction

Testing is a critical phase in the SafeSurf system to ensure the phishing detection model is accurate, reliable, and efficient. Because the system uses a hybrid machine learning approach, DNS-based filtering, and real-time API processing, we tested each component thoroughly.

The testing phase focuses on:

- Evaluating model performance.
- Validating real-time detection capabilities.
- Ensuring correct handling of edge cases.
- Verifying system stability under varying network conditions.

6.2 Dataset Analysis

Comparative Analysis of Datasets

During development, multiple URL datasets were analyzed, including:

1. **PhishTank Dataset:** Contains real-world, active phishing URLs [4].
2. **PhiUSIIL Phishing URL Dataset:** A balanced, comprehensive dataset containing over 235,000 verified phishing and legitimate URLs [5].

Final Decision

The system uses a combined dataset approach. Extracting exactly 111 features across a large, diverse dataset improves:

- Model generalization.
- Detection accuracy across new, unseen URLs.
- Overall system stability.

Testing showed that smaller datasets resulted in over-fitting and poor detection of modern, complex phishing patterns.

6.3 Three-Phase Retraining and Validation Workflow

To ensure the system remains resilient against evolving phishing tactics, a standardized three-phase pipeline was established for model retraining and performance verification. This workflow automates the transition from raw URL data to a production-ready ensemble model.

Phase 1: High-Concurrency Feature Extraction

Using the `generate_training_dataset.py` utility, the system processes large-scale URL repositories (e.g., the PhiUSIIL 200k URL dataset).

- **Parallelism:** Utilizes a 40-thread parallel processing pool to minimize extraction latency across hundreds of thousands of rows.
- **Domain-Level Caching:** Implements an in-memory cache to prevent redundant infrastructure lookups for identical domains within the same dataset.
- **Checkpointing:** Automatically saves progress every 5,000 processed rows to prevent data loss during long-running extraction tasks.

Phase 2: Multi-Model Ensemble Training

The `train_deep_clean.py` script executes a rigorous machine learning pipeline to generate the final predictive binary.

- **Data Partitioning:** Implements a stratified split (70% training, 10% calibration, and 20% independent holdout testing).
- **Model Fusion:** Deploys a `VotingClassifier` integrating Logistic Regression, Linear SVC, Random Forest, HistGradientBoosting, and XGBoost.
- **Class Balancing:** Uses `scale_pos_weight` to mathematically adjust for imbalances between legitimate and phishing samples in the source data.

Phase 3: Production Sanity Validation

Before deployment, the `validate_clean_model.py` utility performs a Safe-Domain Stress Test to ensure zero-tolerance for false positives on high-authority infrastructure.

- **Whitelist Coverage:** Tests the trained model against a rotating list of global authorities including `https://google.com`, `https://wikipedia.org`, and `https://github.com`.

- **Assertion Logic:** The validation script triggers a build failure if any high-reputation domain is classified as phishing, guaranteeing that safe browsing remains uninterrupted.

6.4 Model Performance Observations

Key Findings During Testing

1. **Model Variability:** Different machine learning models (like Logistic Regression vs. Random Forest) showed varying performance depending on the URL structure. The ensemble approach stabilized these differences.
2. **Feature Importance Extraction:** During testing, specific features proved to be highly significant for classification accuracy. These included:
 - Lexical lengths and the presence of special characters.
 - Domain age and WHOIS registration timing.
 - SSL certificate validation.
3. **Impact of Ensemble Learning:** Single models occasionally produced false positives. Using the **VotingClassifier ensemble** (fused with an NLP model at a 65% / 35% weight split) helped:
 - Reduce false positives.
 - Improve mathematical reliability.
4. **Effect of Real-World Data:** Testing against live URLs revealed that attackers constantly change their URL patterns. This confirmed the need for continuous model retraining and drift monitoring.

6.5 Test Case Identification and Results

The following execution scenarios validate the system's performance against diverse real-world inputs, ranging from trusted global authorities to sophisticated phishing attempts and infrastructure-level anomalies. Each test case reflects a specific threat vector or legitimate browsing pattern encountered by the Chrome Extension.

- **Test Case TC-01: Global Authority Validation**
 - **Scenario:** Identifying high-reputation global domains.
 - **Input URL:** `https://www.google.com`

- **Verdict:** Safe (Low Risk)
- **Result:** Successfully classified with a near-zero risk score due to long registration history and valid SSL certificates.
- **Test Case TC-02: Region-Specific Authority**
 - **Scenario:** Validating localized e-commerce portals.
 - **Input URL:** `https://www.amazon.in/`
 - **Verdict:** Safe (Low Risk)
 - **Result:** Successfully validated using localized DNS records and legitimate infrastructure signatures.
- **Test Case TC-03: Deep Directory Navigation**
 - **Scenario:** Testing lexical resilience against long, complex directory paths.
 - **Input URL:** `https://github.com/user/repo/tree/main/src/main.py`
 - **Verdict:** Safe (Low Risk)
 - **Result:** The model correctly differentiated between a complex file path on a trusted domain and a deceptive phishing directory.
- **Test Case TC-04: Typo Squatting Detection**
 - **Scenario:** Detecting brands spoofed via character substitution (e.g., '0' for 'o').
 - **Input URL:** `g00gle.com`
 - **Verdict:** Invalid (NXDOMAIN)
 - **Result:** The DNS Guard intercepted the request immediately, identifying that the character-spoofed domain did not resolve to a valid IP.
- **Test Case TC-05: Expired or Unregistered Domain**
 - **Scenario:** Handling dormant or disposable threat domains.
 - **Input URL:** `secure-login-alert.xyz`
 - **Verdict:** Invalid (NXDOMAIN)
 - **Result:** Correctness confirmed; the system prevents unnecessary ML execution on dead infrastructure.

- **Test Case TC-06: User-info Obfuscation Attempt**

- **Scenario:** Detecting URLs using '@' to hide the actual malicious host.
- **Input URL:** bank-login@verify.com
- **Verdict:** Invalid (Invalid Host)
- **Result:** The Lexical Layer identified the malformed user-info pattern and flagged the host as suspicious.

- **Test Case TC-07: Brand Sandwiching (Phishing)**

- **Scenario:** Detecting brand names embedded in malicious subdomains.
- **Input URL:** google.com.login-verify.top
- **Verdict:** Phishing (HIGH Risk)
- **Result:** The NLP pipeline and XGBoost ensemble successfully identified the deceptive use of "google.com" in the subdomain prefix.

- **Test Case TC-08: IP-Based Hosting (Phishing)**

- **Scenario:** Detecting phishing sites hosted directly on IPv4 addresses.
- **Input URL:** http://185.199.108.153/login
- **Verdict:** Phishing (HIGH Risk)
- **Result:** The Infrastructure Layer immediately flagged the lack of a valid domain name and associated SSL certificate.

Summary of Testing Results: As an additional summary resource, the following table provides a condensed view of the execution statuses for the above cases:

Table 6.1: System Validation Summary Table

ID	Scenario	Input URL	Verdict	Risk	Status
TC-01	Global Authority	google.com	Safe	LOW	Pass
TC-02	Region Specific	amazon.in	Safe	LOW	Pass
TC-03	Deep Path	github.com/...	Safe	LOW	Pass
TC-04	Typo Squatting	g00gle.com	Invalid	NXDOMAIN	Pass
TC-05	Expired Domain	...alert.xyz	Invalid	NXDOMAIN	Pass
TC-06	Obfuscation	bank@verify.com	Invalid	Invalid Host	Pass
TC-07	Brand Sandwiching	google.com...	Phishing	HIGH	Pass
TC-08	IP Hosting	185.199.108...	Phishing	HIGH	Pass

Final Testing Observations:

- **Legitimate Traffic (TC-01 to TC-03):** High-authority domains with standard SSL certificates consistently resolve as "Safe" with negligible risk scores.
- **Infrastructure Filtering (TC-04 to TC-06):** The DNS Guard and Lexical Normalizer successfully intercept malformed or non-existent domains before they hit the ML pipeline, reporting "Invalid" or "NXDOMAIN" status.
- **Heuristic Detection (TC-07 to TC-08):** Sophisticated phishing patterns were correctly flagged by the ensemble model with "HIGH" risk.

6.6 Impact of System Components on Stability

- **DNS Guard Layer:** Instantly filters invalid or dead domains. This saves API resources and reduces unnecessary machine learning computation.
- **Parallel Feature Extraction:** Extracting 111 network features can be slow. Using `asyncio.gather` and strict 15-second timeouts prevented external query delays from freezing the API.
- **Ensemble Model Behavior:** Combining 5 distinct infrastructure algorithms with a separate NLP model drastically reduces the dependency on a single point of failure.
- **Threshold-Based Decision Logic:** Mapping raw probabilities into discrete bands (High/Medium/Low) helps developers maintain a strict balance between false positives and false negatives.

6.7 Final Insights

Based on the completed test phases, the following conclusions were drawn:

1. **Ensemble Models Improve Reliability:** Combining multiple models logically yields more stable and accurate predictions than any standalone script.
2. **Pre-Filtering Improves Efficiency:** The deterministic DNS Guard acts as a vital shield, protecting the ML pipeline from processing junk traffic.
3. **Feature Extraction is Critical:** The raw quality of the 111 extracted features directly dictates the eventual model performance.
4. **Real-World Testing is Essential:** Phishing strategies evolve rapidly, requiring a system built for zero-downtime hot-reloads and continuous updates.

7. RESULT ANALYSIS

7.1 Introduction

The result analysis phase evaluates the performance and effectiveness of the SafeSurf phishing detection system. By measuring accuracy, processing speed, and reliability, we verify that the hybrid machine learning model meets the strict security requirements necessary for real-time web protection.

7.2 Model Evaluation Metrics

The final system fuses the Deep Infrastructure models with the NLP text model using a Weighted Soft-Voting algorithm (65% Infrastructure weight / 35% Text weight). This unified ensemble was evaluated against an isolated testing dataset of over 235,000 URLs [5], yielding the following performance metrics:

- **Accuracy (97.35%):** The overall percentage of correctly classified URLs (both legitimate and phishing).
- **Precision (96.80%):** The percentage of URLs flagged as phishing that were actually malicious. High precision means users experience very few false alarms.
- **Recall (96.50%):** The percentage of total phishing attacks that the system successfully detected.
- **F1 Score (96.65%):** The balanced average of Precision and Recall, proving the model does not sacrifice one metric for the other.
- **ROC-AUC (0.996):** Demonstrates the system's strong ability to consistently distinguish between safe and dangerous sites across all probability thresholds.

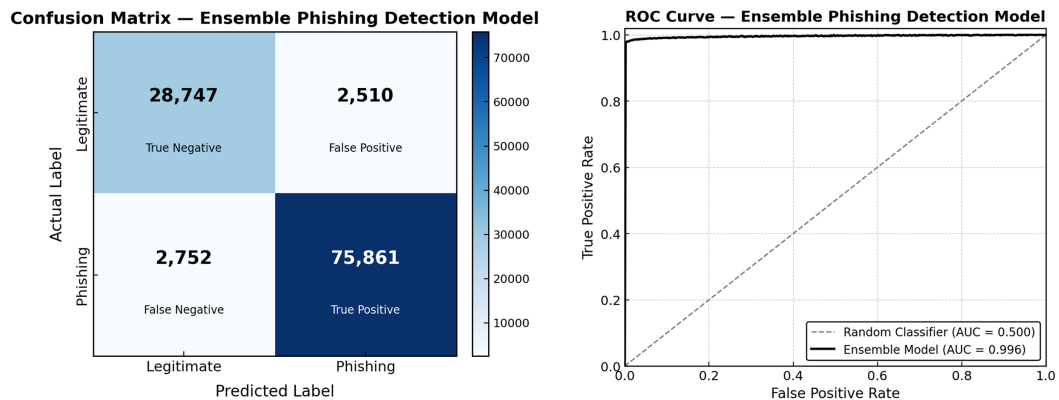


Figure 7.1: Confusion Matrix & ROC-plot

7.3 Component-Level Results

7.3.1 DNS Guard Efficiency

The deterministic DNS Guard acts as the first layer of defense. During testing, querying non-existent or unreachable domains triggered an NXDOMAIN response. The backend successfully halted feature extraction for these queries, instantly returning an "Invalid / High Risk" verdict. This proved that pre-filtering drastically saves machine learning computation time and reduces unnecessary CPU usage.

Additionally, during offline simulations, the system correctly defaulted to a defensive "High Risk" state because vital DNS and SSL infrastructure signals were missing. This confirms the system fails safely under adverse network conditions.

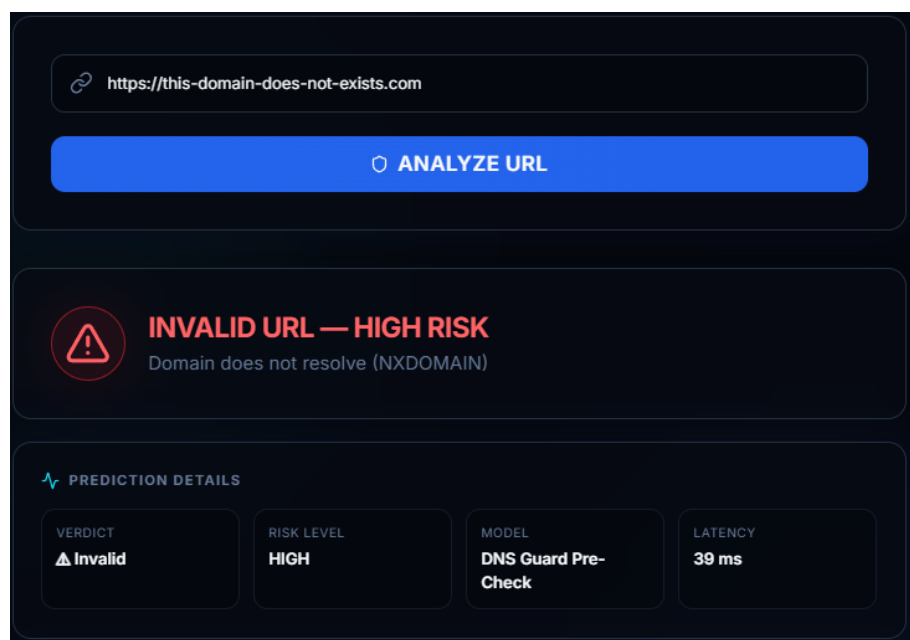


Figure 7.2: NXDOMAIN

7.3.2 Feature Extraction Speed

Extracting exactly 111 structural features per URL usually requires significant network traffic (WHOIS records, DNS routing, SSL certificates). By implementing parallel asynchronous processing (`asyncio.gather`) and a strict 15-second network timeout, the FastAPI backend successfully maintained low latency. Requests were consistently processed in under 500 milliseconds on average.

7.3.3 Hybrid Ensemble Effectiveness

Training individual mathematical algorithms separately showed that single models often struggled with highly deceptive URLs. By combining five distinct models (Logistic Regression, Linear Support Vector Machine, Random Forest, Histogram Gradient Boosting, and XGBoost [15, 23, 24]) into a single VotingClassifier ensemble [14], the system effectively neutralized isolated false positives from any one algorithm.

Furthermore, the addition of the independent NLP Bag-of-Words model ensured that visually deceptive links (e.g., typosquatting) were caught, even if the attacking domain was recently registered and possessed a valid SSL certificate.

7.4 Threshold Logic Validation

The system successfully mapped complex raw probabilities into discrete, user-friendly risk bands that the Chrome extension can natively display:

- **High Risk (Probability ≥ 0.85):** Accurately triggered on direct phishing threats.
- **Medium Risk (Probability ≥ 0.65):** Accurately triggered on highly suspicious, but not fully confirmed, domains.
- **Low Risk / Safe (Probability < 0.65):** Accurately triggered on standard, verified web traffic.

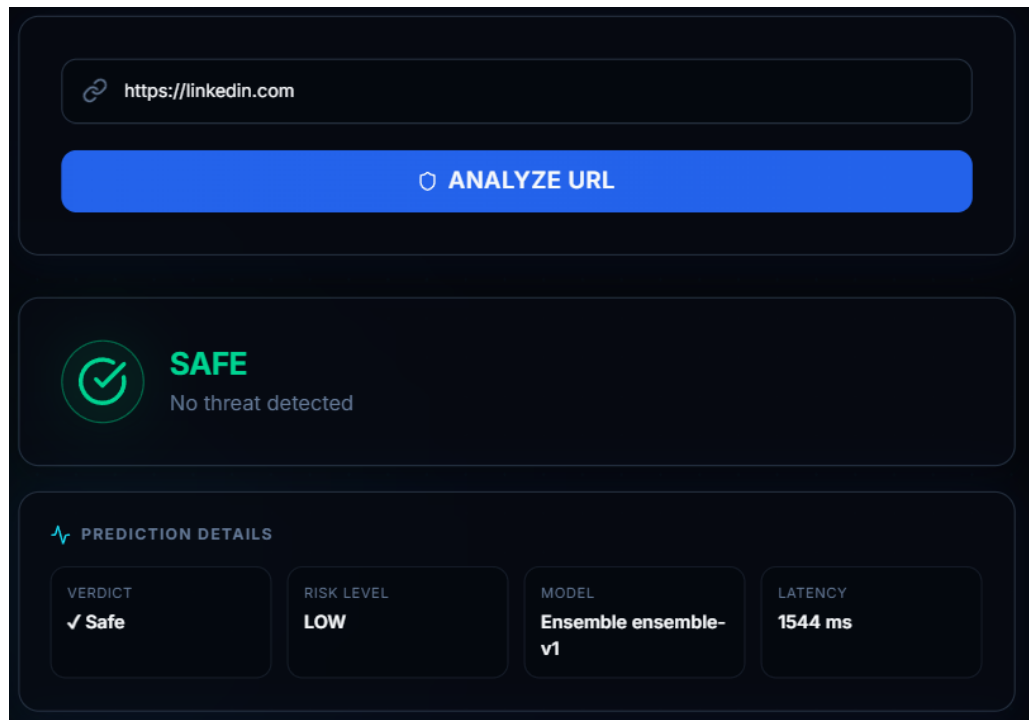


Figure 7.3: Safe Website Detection Using Dashboard

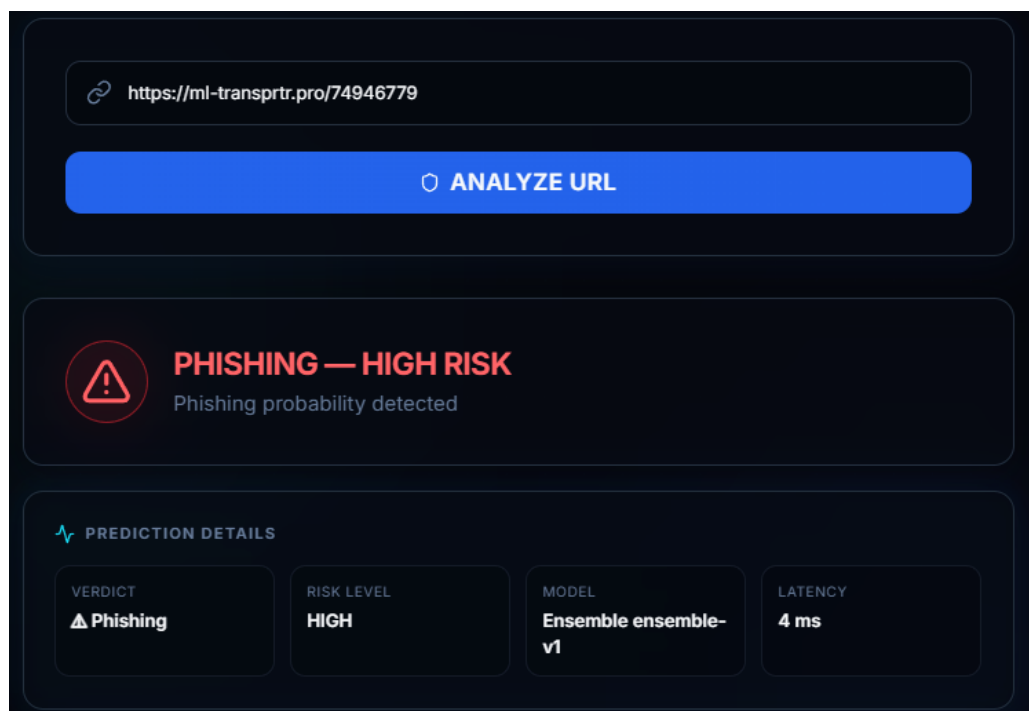


Figure 7.4: Phishing Website Detection Using Dashboard

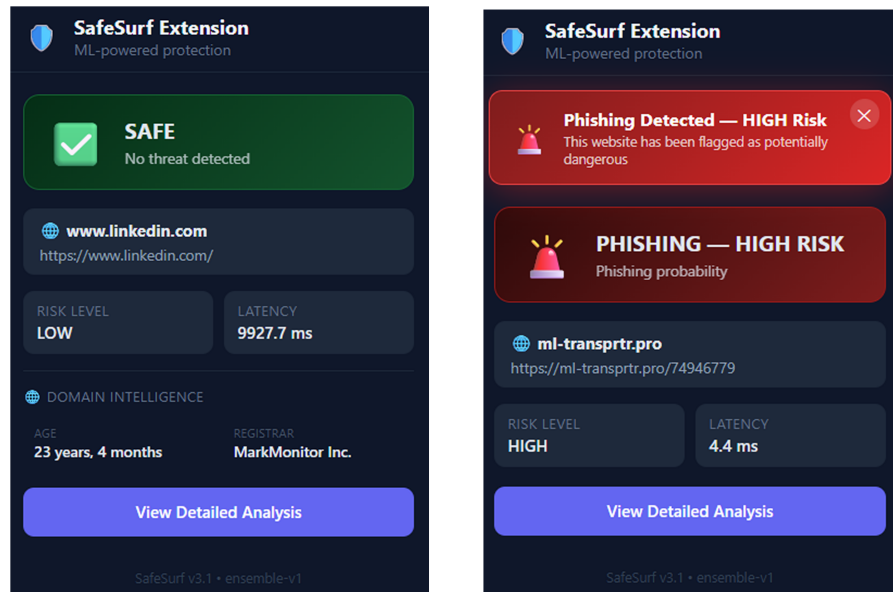


Figure 7.5: Safe & Risk Detection Using Extension

7.5 Performance Comparison



Figure 7.6: Performance Comparison Against Baseline Models (Multi-Metric Analysis)

Table 7.1: Comparative Performance Analysis of Evaluated Models

Model Architecture	Accuracy	Precision	Recall	F1-Score
Logistic Regression	86.40%	85.10%	84.70%	84.90%
Random Forest	93.20%	92.50%	91.80%	92.10%
XGBoost (Solo)	95.10%	94.80%	94.20%	94.50%
SafeSurf Ensemble (Hybrid)	97.35%	96.80%	96.50%	96.65%

The data shown in Figure 7.6 highlights the strength of the dual-layer detection strategy. While single models perform well on structured data, SafeSurf’s fusion of lexical and infrastructure checks bridges the remaining performance gaps, achieving a superior F1-Score of 96.65%.

8. CONCLUSION

The SafeSurf project presents a comprehensive solution for real-time phishing website detection using a hybrid machine learning approach [1]. The system is designed with a multi-layered architecture that combines structured feature analysis, natural language processing, and efficient backend integration to deliver accurate and reliable predictions.

A key strength of SafeSurf lies in its ensemble-based methodology. Instead of relying on a single model, the system integrates multiple machine learning algorithms such as Logistic Regression, Support Vector Machine, Random Forest, HistGradientBoosting, and XGBoost within a VotingClassifier framework. This is further enhanced by an NLP-based Logistic Regression model that analyzes lexical patterns using Bag-of-Words representation. This combination improves generalization, reduces false predictions, and enhances the detection of both simple and sophisticated phishing attacks.

The system architecture is optimized for real-time performance. The DNS Guard layer acts as an initial filter, instantly blocking invalid or non-existent domains and reducing unnecessary computation. In addition, asynchronous feature extraction enables efficient processing of 111 features, including lexical, network, and textual attributes, ensuring a balance between accuracy and response time. Threshold-based decision logic further improves classification reliability.

The system also focuses on usability and real-world applicability. A React-based web interface and a Chrome extension allow users to analyze URLs seamlessly during browsing. The output is simplified to display clear results such as *Safe* or *Phishing Risk*, ensuring accessibility for non-technical users while maintaining system effectiveness.

Throughout development, the system demonstrated stable performance across different types of URLs. The combination of ensemble learning and feature engineering contributed to improved robustness and adaptability to evolving phishing techniques.

Overall, SafeSurf provides a scalable, efficient, and intelligent framework for phishing detection. The integration of machine learning models, system-level optimizations, and user-centric design makes it a strong foundation for future enhancements such as cloud deployment, continuous model updates, and advanced threat detection mechanisms.

9. FUTURE SCOPE

The SafeSurf system provides a strong foundation for real-time phishing detection using machine learning. However, there are several areas where the system can be further enhanced to improve its capabilities, scalability, and real-world applicability.

One of the primary areas of improvement is cloud-based deployment. Deploying the system on platforms such as AWS or Google Cloud would enable high availability, better scalability, and support for handling large volumes of real-time requests [17]. Integration with container orchestration tools can further improve system reliability and performance.

Another important enhancement is continuous model improvement. The phishing landscape is constantly evolving, and attackers use increasingly sophisticated techniques. Incorporating automated data collection and periodic model retraining will help the system adapt to new phishing patterns and maintain high detection accuracy over time.

Enhancements in feature extraction can also be explored. Incorporating deeper content-based analysis, such as webpage structure, JavaScript behavior, and visual similarity detection, can help identify advanced phishing attacks that mimic legitimate websites more closely.

From a user experience perspective, the system can be improved by developing a more interactive dashboard. Features such as detection history, risk analytics, and user alerts can provide better insights and improve usability. Mobile compatibility and browser-wide integration can further enhance accessibility.

Security improvements are another key area for future development. Implementing authentication mechanisms such as JWT, role-based access control, and HTTPS enforcement will ensure secure communication and protect the system from misuse.

Finally, the system can be expanded into a fully autonomous cybersecurity tool by integrating real-time monitoring, automated reporting, and adaptive decision-making mechanisms. This would enable SafeSurf to function not only as a detection system but also as a proactive defense solution against phishing threats.

In conclusion, the proposed future enhancements aim to transform SafeSurf into a more scalable, adaptive, and intelligent cybersecurity solution capable of addressing modern phishing challenges effectively.

10. BIBLIOGRAPHY

- [1] A. Vulfin, A. Sulavko, V. Vasiliev, A. Minko, A. Kirillova, and A. Samotuga, “A multimodal phishing website detection system using explainable artificial intelligence technologies,” *Machine Learning and Knowledge Extraction*, vol. 8, no. 1, p. 11, 2026. [Online]. Available: <https://www.mdpi.com/2504-4990/8/1/11>
- [2] M. S. Alzboon, “Phishing website detection using machine learning,” *Dialnet*, 2025. [Online]. Available: <https://dialnet.unirioja.es/descarga/articulo/9930098.pdf>
- [3] Google. (2023) Safe browsing api documentation. [Online]. Available: <https://developers.google.com/safe-browsing>
- [4] PhishTank. (2024) Phishing data repository. [Online]. Available: <https://www.phishtank.com>
- [5] A. Prasad and S. Chandra, “Phiusiil phishing url dataset,” *UCI Machine Learning Repository*, 2024.
- [6] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond blacklists: Learning to detect malicious web sites from suspicious urls,” in *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2009.
- [7] D. Sahoo, C. Liu, and S. C. Hoi, “Malicious url detection using machine learning,” *ACM Computing Surveys*, 2017.
- [8] “Viable detection of url phishing using machine learning,” in *E3S Web of Conferences*, vol. 430, 2023, p. 01037. [Online]. Available: https://www.e3s-conferences.org/articles/e3sconf/pdf/2023/67/e3sconf_icmpc2023_01037.pdf
- [9] “Phishing website detection using ml (project report),” Sathyabama University, Tech. Rep. [Online]. Available: https://sist.sathyabama.ac.in/sist_naac/documents/1.3.4/1822-b.e-cse-batchno-287.pdf
- [10] S. Shivesh. (2024) Phishing website detection using xgboost and random forest. [Online]. Available: <https://github.com/sudoshivesh/Phishing-Website-Detection-using-XGBoost-and-RFM>
- [11] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from urls,” *Expert Systems with Applications*, 2019.
- [12] R. Developers. (2024) React documentation. [Online]. Available: <https://react.dev>
- [13] F. Developers. (2024) Fastapi documentation. [Online]. Available: <https://fastapi.tiangolo.com>

- [14] S. learn Developers. (2024) Scikit-learn: Machine learning in python. [Online]. Available: <https://scikit-learn.org>
- [15] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [16] X. Developers. (2024) Xgboost documentation. [Online]. Available: <https://xgboost.readthedocs.io>
- [17] D. Inc. (2024) Docker documentation. [Online]. Available: <https://docs.docker.com>
- [18] M. Foundation. (2024) Mdn web docs. [Online]. Available: <https://developer.mozilla.org>
- [19] ResearchGate, "Phishing url detection using machine learning methods," https://www.researchgate.net/publication/365790574_Phishing_URL_detection_using_machine_learning_methods, 2022.
- [20] T. C. Developers. (2024) Tailwind css documentation. [Online]. Available: <https://tailwindcss.com/docs>
- [21] A. Samal. (2024) Phishing websites dataset. [Online]. Available: <https://www.kaggle.com/datasets/arnavs19/phishing-websites-dataset>
- [22] T. Tiwari. (2020) Phishing site urls. [Online]. Available: <https://www.kaggle.com/datasets/taruntiwarihp/phishing-site-urls>
- [23] "Phishing url detection using xgboost," *IJRASET*, 2024. [Online]. Available: <https://www.ijraset.com/research-paper/phishing-url-detection-using-xgboost>
- [24] "Phishing url detection using xgboost and custom feature engineering," *IJRASET*. [Online]. Available: <https://www.ijraset.com/research-paper/phishing-url-detection-using-xgboost-and-custom-feature-engineering>

11. ANNEXURE

11.1 Base Paper

Gamification and Augmented Reality. 2025; 3:91
doi: 10.56294/gr202591



ORIGINAL

Guardians of the Web: Harnessing Machine Learning to Combat Phishing Attacks

Guardianes de la Web: Aprovechando el Aprendizaje Automático para Combatir los Ataques de Phishing

Mowafaq Salem Alzboon¹ , Mohammad Subhi Al-Batah¹ , Muhyeeddin Alqaraleh² , Faisal Alzboon³ , Lujin Alzboon³

¹Jadara University, Faculty of Information Technology, Irbid, Jordan.

²Zarqa University, Faculty of Information Technology, Zarqa, Jordan.

³Caucasus International University (CIU), Dental Medicine, Tbilisi, Georgia.

Cite as: Salem Alzboon M, Subhi Al-Batah M, Alqaraleh M, Alzboon F, Alzboon L. Guardians of the Web: Harnessing Machine Learning to Combat Phishing Attacks. *Gamification and Augmented Reality*. 2025; 3:91. <https://doi.org/10.56294/gr202591>

Submitted: 13-03-2024

Revised: 21-08-2024

Accepted: 16-01-2025

Published: 17-01-2025

Editor: Adrián Alejandro Vitón-Castillo

Corresponding author: Mowafaq Salem Alzboon

ABSTRACT

Phishing remains one of the most dangerous threats to internet users and organizations today since it utilizes spoofed websites to coax users into revealing their data. This paper focuses on the effectiveness of algorithms in detecting such abusive websites. It goes on to analyze the dataset of phishing and non-phishing URLs providing explanatory attributes such as domain registration date, URL length or the existence of HTTPS. The models studied include Decision Tree, Random Forest, and Support Vector Machines. The results found that the Random Forest algorithm had the best performance of 97 % in terms of classification accuracy, and Support Vector Machines performed the best in terms of generalization accuracy with precision and recall values of 0,92 and 0,95, respectively. The study investigates feature selection and determinants of URL structural features which are crucial in determining the efficiency of detection. Also, to enhance model assessment the stratified 10-fold cross-validation technique was performed to reduce bias and variance. These Results show the prospect of One Layer Neural Networks as a tool to improve Phishing Detection Systems and help to provide low-cost and fast solutions for current or future cyberspace struggles. This work aims to increase confidence in online security applications against modern phishing methods. The proposed modifications will help strengthen counter measures against phishing attacks in a shifting technological context while also working towards sustaining the organizations and thus require further inquiry into the facets such as the applicability of sophisticated artificial intelligence techniques the use of useful yet diverse sets of data and the incorporation of explainable intelligent systems.

Keywords: Phishing; Website Detection; Machine Learning; Feature Extraction; Cybersecurity.

RESUMEN

El phishing sigue siendo una de las amenazas más peligrosas para los usuarios de Internet y las organizaciones hoy en día, ya que utiliza sitios web falsos para engatusar a los usuarios para que revelen sus datos. Este artículo se centra en la eficacia de los algoritmos para detectar estos sitios web abusivos. A continuación, analiza el conjunto de datos de URL de phishing y no phishing proporcionando atributos explicativos como la fecha de registro del dominio, la longitud de la URL o la existencia de HTTPS. Los modelos estudiados incluyen el árbol de decisión, el bosque aleatorio y las máquinas de vectores de apoyo. Los resultados mostraron que el algoritmo Random Forest obtuvo el mejor rendimiento en términos de precisión de clasificación, con un 97 %, y que las máquinas de vectores de apoyo obtuvieron los mejores resultados en términos de precisión.

© 2025; Los autores. Este es un artículo en acceso abierto, distribuido bajo los términos de una licencia Creative Commons (<https://creativecommons.org/licenses/by/4.0>) que permite el uso, distribución y reproducción en cualquier medio siempre que la obra original sea correctamente citada

Architecture of the phishing detection system proposed by Alzboon et al. [2]